

Automated Program Reasoning — Some Ideas —

Christoph Benz Müller

Freie Universität Berlin | University of Luxembourg



TU Vienna — October 22, 2019

Automated Program Reasoning — Some Ideas —

Christoph Benz Müller

Freie Universität Berlin | University of Luxembourg



TU Vienna — October 22, 2019

Automated Program Reasoning — Some Ideas —

Christoph Benzmüller

Freie Universität Berlin | University of Luxembourg



TU Vienna — October 22, 2019

Eine Fallstudie zur Spezifikation von
Systemanforderungen in der Spezifikationssprache
OBSCURE

Diplomarbeit von Christoph Benzmüller

Angefertigt nach einem Thema von Prof. Dr.-Ing. Loeckx am Fachbereich 14,
Informatik, der Universität des Saarlandes, Saarbrücken.

Diese Diplomarbeit wurde erstellt in Zusammenarbeit mit dem
Fraunhofer-Institut Biomedizinische Technik (IBMT) in St. Ingbert
unter der Leitung von Prof. Dr. med. Gersonde.

5. Juli 1994

Inhaltsverzeichnis

Verzeichnis der Abbildungen	3
1 Einleitung	5
1.1 Motivation einer formalen Software-Entwicklung	5
1.2 OBSCURE	8
1.2.1 Die Spezifikationsprache OBSCURE	8
1.2.2 Die Spezifikationsumgebung OBSCURE-System	9
1.3 SUNRISE	9
1.4 Überblick zu den einzelnen Kapiteln	11
1.5 Anmerkungen zur Form	11
2 Systemanalyse	14
2.1 Strategie, Begriffe und Beschreibungsmittel	15
2.2 Analyse des SUNRISE-Systems	21
2.2.1 SUNRISE-Systemumwelt	21
2.2.2 SUNRISE-Systemziele	24
2.2.3 Externe Ereignisse von SUNRISE	26
2.2.4 Essentielle Aktivitäten von SUNRISE	27
2.2.5 Essentielle Informationen von SUNRISE	27
2.2.6 Problemhinweise	30
2.3 Der Ausschnitt <i>duales Speicherkonzept</i>	31
2.3.1 Ein Beispiel	33
2.3.2 Entity-Relationship-Diagramm zum Ausschnitt <i>duales Speicherkonzept</i>	36
2.3.3 Attribute der Entitäten	38
2.3.4 Datenflusdiagramme zur essentiellen Aktivität <i>Wav</i>	44
2.4 Eine Sicherheitsansage	48
2.5 Anmerkungen und Namenskonventionen	50
3 Formale Anforderungsspezifikation	52
3.1 Gründe für eine formale Spezifikation	53
3.2 Struktur der Spezifikation	53
3.3 Spezifikation der Datenmodellebene	54
3.4 Spezifikation der Aktivitätenebene	60

Formal Specification (and a little Verification) — already in my Diploma Thesis

Inhaltsverzeichnis

Verzeichnis der Abbildungen	3
1 Einleitung	5
1.1 Motivation einer formalen Software-Entwicklung	5
1.2 OBSCURE	8
1.2.1 Die Spezifikationsprache OBSCURE	8
1.2.2 Die Spezifikationsumgebung OBSCURE-System	9
1.3 SECURE	9
1.4 Überblick zu den einzelnen Kapiteln	11
1.5 Anmerkungen zur Form	11
2 Systemanalyse	14
2.1 Strategie, Begriffe und Beschreibungsmittel	15
2.2 Analyse des SECURE-Systems	21
2.2.1 SECURE-Systemanwahl	21
2.2.2 SECURE-Systemziele	24
2.2.3 Externe Ereignisse von SECURE	26
2.2.4 Essentielle Aktivitäten von SECURE	27
2.2.5 Essentielle Informationen von SECURE	27
2.2.6 Problemhinweise	30
2.3 Der Ausschnitt duales Speicherkonzept	31
2.3.1 Ein Beispiel	33
2.3.2 Entity-Relationship-Diagramm zum Ausschnitt duales Speicherkonzept	36
2.3.3 Attribute der Entitäten	38
2.3.4 Datendflussdiagramme zur essentiellen Aktivität WNAV	44
2.4 Eine Sicherheitsansage	48
2.5 Anmerkungen und Namenskonventionen	50
3 Formale Anforderungsspezifikation	52
3.1 Gründe für eine formale Spezifikation	53
3.2 Struktur der Spezifikation	53
3.3 Spezifikation der Datenmodellebene	54
3.4 Spezifikation der Aktivitätsebene	60

INHALTSVERZEICHNIS

3.4.1 Spezifikation von Bilddaten	61
3.4.2 Spezifikation der Integritätsbedingungen	63
3.4.3 Spezifikation der essentiellen Aktivität WNAV	65
3.4.4 Zusammenfassung der Aktivitätsebene	71
3.5 Rapid-Prototyping	76
3.5.1 Erreichbare Datenbankzustände	76
3.5.2 Allgemeine Datenbankzustände	77
3.5.3 Integre Datenbankzustände	77
3.5.4 Anmerkungen zu erreichbaren und integren Datenbankzuständen	77
3.5.5 Ein Beispiel: Ausgangszustand zur Anwendung von WNAV	79
3.5.6 Ein Beispiel: Resultatzustand einer Anwendung von WNAV	79
4 Zusammenhang zur aktuellen Implementierung von SECURE	81
4.1 Begriff Systementwurf	82
4.2 Informale Erläuterungen zur aktuellen Implementierung	82
4.2.1 Ein Beispiel: Ausgangszustand einer Anwendung von WNAV	83
4.2.2 Ein Beispiel: Resultatzustand einer Anwendung von WNAV	86
4.3 Abbildung der formalen Spezifikation auf Bestandteile der Implementierung	88
5 Sicherheits-Güteeigenschaften	92
5.1 Überblick zu den IT-Sicherheitskriterien ITSEC	93
5.2 Einordnung der vorliegenden Arbeit	96
6 Zusammenfassung und Anmerkungen	98
6.1 Zusammenfassung der Arbeit	98
6.2 Anmerkungen zur Systemanalyse	99
6.3 Anmerkungen zu OBSCURE	100
6.4 Anmerkungen zu SECURE	104
6.5 Anmerkungen zum Thema Verifikation	105
6.6 Vorschlag für eine Weiterführung der Fallstudie	106
Anhang	108
A Die Spezifikation der Datenmodellebene	109
A.1 Struktur der Spezifikation	110
A.2 Spezifikation der Attributsebene	111
A.2.1 Domainarten	111
A.2.2 Attributarten	114
A.2.3 Schlüssel	119
A.3 Spezifikation der Datenbankebene	119
A.3.1 Entity-Komponenten	120
A.3.2 Relationship-Komponenten	127
A.3.3 Spezifikation der Datenbankkarte DB	134

Formal Verification of Software and Hardware — Model Checking

- ▶ systematic exploration of finite models, abstraction needed for the infinite
- ▶ properties typically described in temporal logics: linear temporal logic (LTL), property specification language (PSL), computational tree logic (CTL), etc.
- ▶ SAT solving as a key technology for proof automation

Formal Verification of Software and Hardware — Model Checking

- ▶ systematic exploration of finite models, abstraction needed for the infinite
- ▶ properties typically described in temporal logics: linear temporal logic (LTL), property specification language (PSL), computational tree logic (CTL), etc.
- ▶ SAT solving as a key technology for proof automation



DPLL-Algorithmus — Entwicklung am Beispiel

	Eingabe-Formel	(CNF & Initialisierung)
[]	$A\bar{B}, \bar{B}, BCF, C\bar{G}F, \bar{C}FG, \bar{C}\bar{F}G, \bar{D}E, \bar{F}\bar{G}$	(Unit-Propagation)
[$\bar{B}$]	$A\bar{B}, \bar{B}, \cancel{BCF}, C\bar{G}F, \bar{C}FG, \bar{C}\bar{F}G, \bar{D}E, \bar{F}\bar{G}$	(Pure-Literal)
[$\bar{B}\bar{D}$]	$A\bar{B}, \bar{B}, \cancel{BCF}, C\bar{G}F, \bar{C}FG, \bar{C}\bar{F}G, \bar{D}E, \bar{F}\bar{G}$	(Split)
[$\bar{B}\bar{D}C_b$]	$A\bar{B}, \bar{B}, BCF, C\bar{G}F, \cancel{\bar{C}FG}, \cancel{\bar{C}\bar{F}G}, \bar{D}E, \bar{F}\bar{G}$	(Split)
[$\bar{B}\bar{D}C_bF_b$]	$A\bar{B}, \bar{B}, BCF, C\bar{G}F, \bar{C}FG, \cancel{\bar{C}\bar{F}G}, \bar{D}E, \cancel{\bar{F}\bar{G}}$	(Unit-Propagation)
[$\bar{B}\bar{D}C_bF_bG$]	$A\bar{B}, \bar{B}, BCF, C\bar{G}F, \bar{C}FG, \bar{C}\bar{F}G, \bar{D}E, \cancel{\bar{F}\bar{G}}$	(Backtrack)
[$\bar{B}\bar{D}C_b\bar{F}$]	$A\bar{B}, \bar{B}, BCF, C\bar{G}F, \cancel{\bar{C}\bar{F}G}, \bar{C}FG, \bar{D}E, \bar{F}\bar{G}$	(Unit-Propagation)
[$\bar{B}\bar{D}C_b\bar{F}G$]	$A\bar{B}, \bar{B}, BCF, C\bar{G}F, \bar{C}FG, \bar{C}\bar{F}G, \bar{D}E, \bar{F}\bar{G}$	(Solved)

Abstrakter DPLL Algorithmus (Regelbasierte Darstellung)

Wende die Regeln erschöpfend an mit folgender Priorität:

1. Solved
2. Fail
3. Backtrack
4. Unit-Propagation
5. Pure-Literal
6. Split

Formal Verification of Software and Hardware — Deductive Verification

- ▶ system vs. system specification $\rightarrow$ mathematical proof obligations
- ▶ Hoare triples: **{Precondition}** *program* **{Postcondition}**

Formal Verification of Software and Hardware — Deductive Verification

- ▶ system vs. system specification $\rightarrow$ mathematical proof obligations
- ▶ Hoare triples: $\{\mathbf{x} = ? \mathbf{0}\}$ *if* $x = ? 0$ *then* $x := 1$ *else* $x := 2$ $\{\mathbf{x} = ? \mathbf{1}\}$

Formal Verification of Software and Hardware — Deductive Verification

- ▶ system vs. system specification $\rightarrow$ mathematical proof obligations
- ▶ Hoare triples: $\{\mathbf{x} =^? \mathbf{0}\}$ *if* $x =^? 0$ *then* $x := 1$ *else* $x := 2$ $\{\mathbf{x} =^? \mathbf{1}\}$
- ▶ interactive theorem provers: HOL, Isabelle, Coq, Lean, PVS, ACL2, etc.
- ▶ automated theorem provers: CVC4, Z3 (SMT solvers), Vampire, E, Spass (FOF), Leo-III/Leo-II, Satallax, Zipperpin (HOL), etc

Formal Verification of Software and Hardware — Deductive Verification

- ▶ system vs. system specification $\rightarrow$ mathematical proof obligations
- ▶ Hoare triples: $\{x = ? 0\}$ if $x = ? 0$ then $x := 1$ else $x := 2$ $\{x = ? 1\}$
- ▶ interactive theorem provers: HOL, Isabelle, Coq, Lean, PVS, ACL2, etc.
- ▶ automated theorem provers: CVC4, Z3 (SMT solvers), Vampire, E, Spass (FOF), Leo-III/Leo-II, Satallax, Zipperpin (HOL), etc



The screenshot shows the Isabelle website. At the top, there is a logo for Isabelle and logos for the University of Cambridge Computer Laboratory and TUM. A navigation menu on the left includes links for Home, Overview, Installation, Documentation, and Site Mirrors. The main content area has a section titled "What is Isabelle?" which describes it as a generic proof assistant. Below this, a blue banner announces "Now available: Isabelle2019 (June 2019)". There are download buttons for macOS and Linux/Windows. A section titled "Some notable changes:" lists improvements like the DejaVu font collection, virtual file-system access, and HOL export code.

Isabelle UNIVERSITY OF CAMBRIDGE Computer Laboratory TUM TECHNISCHE UNIVERSITÄT MÜNCHEN

What is Isabelle?

Isabelle is a generic proof assistant. It allows mathematical formulas to be expressed in a formal language and provides tools for proving those formulas in a logical calculus. Isabelle was originally developed at the University of Cambridge and Technische Universität München, but now includes numerous contributions from institutions and individuals worldwide. See the [Isabelle overview](#) for a brief introduction.

Now available: Isabelle2019 (June 2019)

Download for macOS

[Download for Linux](#) - [Download for Windows](#) - [Download for macOS](#)

Some notable changes:

- Improved Isabelle/DejaVu font collection, suitable for text and GUI
- Various Isabelle/Edt improvements, with virtual file-system access to sessions and exports
- Improved headless PIDE session (and server)
- HOL export code now generates logical files in the theory and session context, e.g. browsable as `Isabelle-exports.in`



The screenshot shows the Leo III website. At the top, there is a logo for Leo III and logos for Freie Universität Berlin and the University of Cambridge. A navigation menu includes links for Overview, People, Publications, and Theses. The main content area has a section titled "Leo III - A MASSIVELY PARALLEL HIGHER-ORDER THEOREM PROVER". Below this, a news item titled "Leo-III wins the CASC LTB division!" is displayed, along with a section titled "Overview" which describes the system's architecture and its success in the CASC LTB division.

Leo III Freie Universität Berlin

Leo III - A MASSIVELY PARALLEL HIGHER-ORDER THEOREM PROVER

Overview **People** **Publications** **Theses**

News

Leo-III wins the CASC LTB division!

At the most recent CASC 27, Leo-III 1.8 ranked 1st place in the periodically relevant Large Theory Batch (LTB) division. With over 5400 of 1000 solved problems (54%), Leo-III completes the challenge with approx. 800 problems lead before the next ATP systems, E and Vampire. Also, Leo-III came in second after Delta-e, in the Trif division of CASC.

New Study on Leo-III's effectivity

An independent, large evaluation study identifies Leo-III as the worlds leading and most general higher-order automated theorem prover.

For further details, see the pre-print of [GRUNGE: A \(partial\) ATP Challenge](#) (Chad E. Brown, Thibaut Gauthier, Gregory Katskov, Geoff Sutcliffe, Josef Urban) on arXiv.org.

Overview

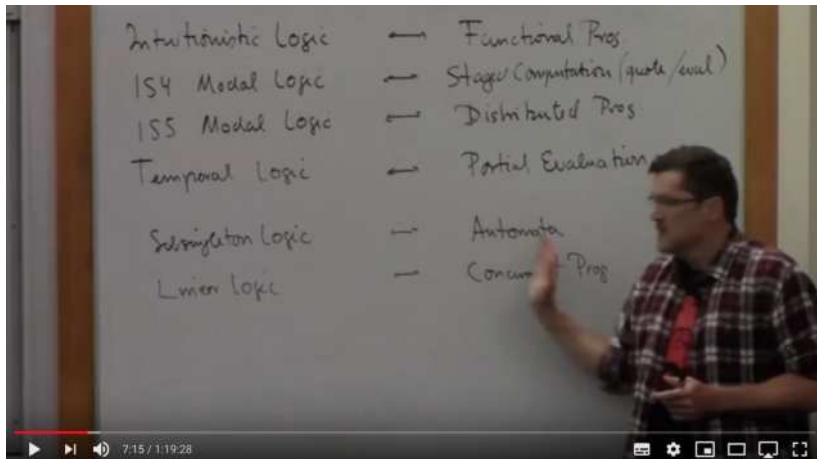
The automated theorem proving systems LEO-I and LEO-II have found international acclaim as very successful reasoners for classical higher order logic. Novel contributions of LEO-I include a native (versus axiomatic) treatment of the extensionality principles and the cooperation with external reasoners (such as the first-order prover E) via a flexible agent architecture. The implementation of LEO-II did significantly influence the parallel

Formal Verification of Software and Hardware — Deductive Verification

- ▶ system vs. system specification $\rightarrow$ mathematical proof obligations
- ▶ Hoare triples: $\{\mathbf{x} = ? \mathbf{0}\}$ if $x = ? 0$ then $x := 1$ else $x := 2$ $\{\mathbf{x} = ? \mathbf{1}\}$
- ▶ interactive theorem provers: HOL, Isabelle, Coq, Lean, PVS, ACL2, etc.
- ▶ automated theorem provers: CVC4, Z3 (SMT solvers), Vampire, E, Spass (FOF), Leo-III/Leo-II, Satallax, Zipperpin (HOL), etc

What logic formalisms?

Logic: the Study of Deductive Inference



Frank Pfenning: Substructural Type Systems and Concurrent Programming, Oregon Programming Languages Summer School, 2017
<https://www.youtube.com/watch?v=ZW-VWVa1o9g>

VERISmart (Discussion Paper): A Language Proposal for Smart Contracts

Christoph Benz Müller and Alexander Steen

February 8, 2019

Abstract

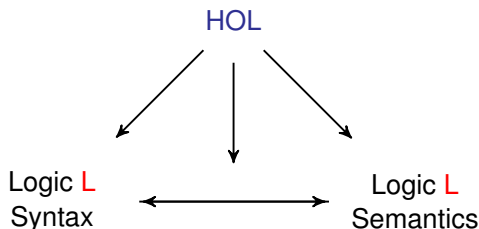
Some ideas towards a verifiable language, data model and execution environment for smart contracts are presented. Our proposal partly results from experiments conducted with a meta-logical approach to universal reasoning in classical higher-order logic (HOL), which we combine with several new suggestions. We discuss smart contract language aspects and also address questions regarding the data model and the execution model.

Motivation: Fully verifiable (deliberately restricted) smart contract language?

We need **combinations** of (**quantified extensions**) of different logics:

- ▶ **Dynamic Logics**—*states, processes, partial evaluation*
- ▶ **Hoare-Triples**—*convenient/intuitive way to organize program reasoning*
- ▶ **Separation Logic**—*accessing shared structures, storage (de-)allocation*
- ▶ **Authorisation Logic**—*authorization in (distributed) computing systems*
- ▶ **Linear Logic**—*resource-boundedness and interaction*
- ▶ (and maybe **Free Logic**—*partiality and undefinedness, in an elegant way*)

Flexible Reasoning Technology: SSEs in HOL



Examples for **L** we have already studied:

Intuitionistic Logics, (Mathematical) Fuzzy Logics, Free Logic, **Multimodal Logics**, Description Logics, Conditional Logics, **Access Control Logics**, Hybrid Logics, Multivalued Logics, Deontic Logics, Paraconsistent Logics, **Computation Tree Logic**, **Dynamic Logic**,...

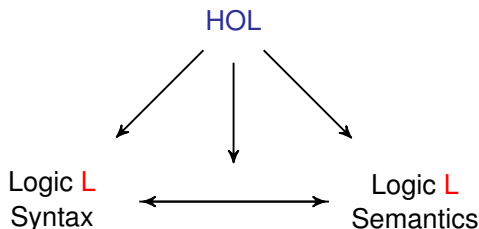
Embedding works also for quantifiers (first-order & higher-order)

HOL provers become universal logic reasoning engines!

interactive: Isabelle/HOL, PVS, HOL4, Hol Light, Coq/HOL, ...

automated: Leo-III, LEO-II, Satallax, TPS, Nitpick, Isabelle/HOL, ...

Flexible Reasoning Technology: SSEs in HOL



Examples for **L we have already studied:**

Intuitionistic Logics, (Mathematical) Fuzzy Logics, Free Logic, **Multimodal Logics**, Description Logics, Conditional Logics, **Access Control Logics**, Hybrid Logics, Multivalued Logics, Deontic Logics, Paraconsistent Logics, **Computation Tree Logic**, **Dynamic Logic**,...

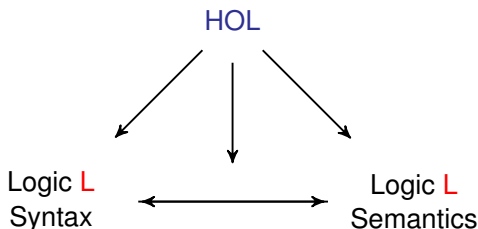
Embedding works also for quantifiers (first-order & higher-order)

HOL provers become universal logic reasoning engines!

interactive: Isabelle/HOL, PVS, HOL4, Hol Light, Coq/HOL, ...

automated: Leo-III, LEO-II, Satallax, TPS, Nitpick, Isabelle/HOL, ...

Flexible Reasoning Technology: SSEs in HOL



Examples for **L we have already studied:**

Intuitionistic Logics, (Mathematical) Fuzzy Logics, Free Logic, **Multimodal Logics**, Description Logics, Conditional Logics, **Access Control Logics**, Hybrid Logics, Multivalued Logics, Deontic Logics, Paraconsistent Logics, **Computation Tree Logic**, **Dynamic Logic**,...

Embedding works also for quantifiers (first-order & higher-order)

HOL provers become universal logic reasoning engines!

interactive: Isabelle/HOL, PVS, HOL4, Hol Light, Coq/HOL, ...

automated: Leo-III, LEO-II, Satallax, TPS, Nitpick, Isabelle/HOL, ...

One Option for Talk: Quantified Multimodal Logics — Wise Men Puzzle

Data in Brief 24 (2019) 103823

Contents lists available at ScienceDirect

Data in brief

journal homepage: www.elsevier.com/locate/dib

Data Article

Universal (meta-)logical reasoning: The Wise Men Puzzle (Isabelle/HOL dataset)



Christoph Benz Müller^{a, b, *}

^a Freie Universität Berlin, Germany
^b University of Luxembourg, Luxembourg

ARTICLE INFO

Article history:
Received 20 November 2018
Received in revised form 22 February 2019
Accepted 4 March 2019
Available online 20 March 2019

Keywords:
Universal logical reasoning;
Automated theorem proving;
Higher-order logic

ABSTRACT

The authors universal (meta-)logical reasoning approach is demonstrated and assessed with a prominent riddle in epistemic reasoning: the Wise Men Puzzle. The presented solution puts a particular emphasis on the adequate modeling of common knowledge and it illustrates the elegance and the practical relevance of the shallow semantical embedding approach when utilized within modern proof assistant systems such as Isabelle/HOL. The contributed dataset provides supporting evidence for claims made in the article "Universal (meta-)logical reasoning: Recent successes" (Benz Müller, 2019).

© 2019 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

But instead, I will show you very recent work on PDL and Hoare-triples in HOL

Propositional Dynamic Logic (PDL) — Syntax (Formulas & Programs)

[Pratt'76][Fisher&Ladner'79]

Given (countable sets): atomic formulas $\mathcal{A}$ and atomic programs $\mathcal{P}$

Complex formulas and complex programs are defined as follows:

Propositional Dynamic Logic (PDL) — Syntax (Formulas & Programs)

[Pratt'76][Fisher&Ladner'79]

Given (countable sets): atomic formulas $\mathcal{A}$ and atomic programs $\mathcal{P}$

Complex formulas and complex programs are defined as follows:

Formulas are defined as the smallest set $\mathbf{F}$ such that:

1. If atomic formula $A \in \mathcal{A}$, then $A \in \mathbf{F}$
2. $\perp \in \mathbf{F}$
3. If $A \in \mathbf{F}$, then $\neg A \in \mathbf{F}$
4. If $A, B \in \mathbf{F}$, then $(A \vee B) \in \mathbf{F}$
5. If program $\alpha \in \mathcal{P}$ and $A \in \mathbf{F}$, then $[\alpha]A \in \mathbf{F}$
— “execution of α in a present state leads to a state where A is True”

Propositional Dynamic Logic (PDL) — Syntax (Formulas & Programs)

[Pratt'76][Fisher&Ladner'79]

Given (countable sets): atomic formulas $\mathcal{A}$ and atomic programs $\mathcal{P}$

Complex formulas and complex programs are defined as follows:

Formulas are defined as the smallest set $\mathbf{F}$ such that:

1. If atomic formula $A \in \mathcal{A}$, then $A \in \mathbf{F}$
2. $\perp \in \mathbf{F}$
3. If $A \in \mathbf{F}$, then $\neg A \in \mathbf{F}$
4. If $A, B \in \mathbf{F}$, then $(A \vee B) \in \mathbf{F}$
5. If program $\alpha \in \mathcal{P}$ and $A \in \mathbf{F}$, then $[\alpha]A \in \mathbf{F}$
— “execution of α in a present state leads to a state where A is True”

Programs are defined as the smallest set $\mathbf{P}$ such that:

1. If atomic program $\alpha \in \mathcal{P}$, then $\alpha \in \mathbf{P}$ — “e.g., assertions treated as atoms”
2. If $\alpha, \beta \in \mathbf{P}$, then $(\alpha; \beta) \in \mathbf{P}$ — “do α followed by β ”
3. If $\alpha, \beta \in \mathbf{P}$, then $(\alpha \cup \beta) \in \mathbf{P}$ — “do α or β , non-deterministically”
4. If $\alpha \in \mathbf{P}$, then $\alpha^* \in \mathbf{P}$ — “repeat α for a non-deterministic number of times”
5. If A is a formula then $A? \in \mathbf{P}$ — “proceed if A is true, else fail”

Propositional Dynamic Logic (PDL) — Semantics (Formulas & Programs)

Semantics of PDL is interpreted over *Labeled Transition Systems* $M = (W, R, V)$.

- W : non-empty set of worlds/states
- R : mapping from $\mathcal{P}$ (atomic programs) to W^2 (binary relations over W)
- V : mapping $\mathcal{A}$ (atomic formulas) to subsets of W (truth-sets)

Complex programs: interpreted as composite binary relations over W

Propositional Dynamic Logic (PDL) — Semantics (Formulas & Programs)

Semantics of PDL is interpreted over *Labeled Transition Systems* $M = (W, R, V)$.

- W : non-empty set of worlds/states
- R : mapping from $\mathcal{P}$ (atomic programs) to W^2 (binary relations over W)
- V : mapping $\mathcal{A}$ (atomic formulas) to subsets of W (truth-sets)

Complex programs: interpreted as composite binary relations over W

$\|\alpha\|$, the **interpretation of a program** $\alpha \in \mathbf{P}$, defined inductively ($\delta \in \mathcal{P}; \beta, \gamma \in \mathbf{P}; A \in \mathbf{F}$):

$$\|\delta\| \quad \equiv \quad R(\delta) \in W^2$$

$$\|\beta; \gamma\| \quad \equiv \quad \{(w, v) \in W^2 \mid \exists u \in W. (w, u) \in \|\beta\| \text{ and } (u, v) \in \|\gamma\|\}$$

$$\|\beta \cup \gamma\| \quad \equiv \quad \|\beta\| \cup \|\gamma\|$$

$$\|\beta^*\| \quad \equiv \quad (\|\beta\|)^* \quad R^* \text{ is reflexive-transitive closure of } R$$

$$\|A?\| \quad \equiv \quad \{(w, w) \in W^2 \mid M, w \models A\}$$

Propositional Dynamic Logic (PDL) — Semantics (Formulas & Programs)

Semantics of PDL is interpreted over *Labeled Transition Systems* $M = (W, R, V)$.

- W : non-empty set of worlds/states
- R : mapping from $\mathcal{P}$ (atomic programs) to W^2 (binary relations over W)
- V : mapping $\mathcal{A}$ (atomic formulas) to subsets of W (truth-sets)

Complex programs: interpreted as composite binary relations over W

$\|\alpha\|$, the **interpretation of a program** $\alpha \in \mathbf{P}$, defined inductively ($\delta \in \mathcal{P}; \beta, \gamma \in \mathbf{P}; A \in \mathbf{F}$):

$$\begin{aligned}\|\delta\| &\equiv R(\delta) \in W^2 \\ \|\beta; \gamma\| &\equiv \{(w, v) \in W^2 \mid \exists u \in W. (w, u) \in \|\beta\| \text{ and } (u, v) \in \|\gamma\|\} \\ \|\beta \cup \gamma\| &\equiv \|\beta\| \cup \|\gamma\| \\ \|\beta^*\| &\equiv (\|\beta\|)^* && R^* \text{ is reflexive-transitive closure of } R \\ \|[A?]\| &\equiv \{(w, w) \in W^2 \mid M, w \models A\}\end{aligned}$$

$M, w \models F$, the **evaluation of formula** F (in M, w), is defined as ($A \in \mathcal{A}; B, C \in \mathbf{F}; \alpha \in \mathbf{P}$):

$$\begin{array}{lll} M, w \models A & \text{iff} & w \in V(A) \\ M, w \models \neg B & \text{iff} & \text{not } M, w \models B \\ M, w \models B \vee C & \text{iff} & M, w \models B \text{ or } M, w \models C \\ M, w \models [\alpha]B & \text{iff} & M, v \models B \text{ for every } v \in W \text{ s.t. } (w, v) \in \|\alpha\| \end{array}$$

Propositional Dynamic Logic (PDL) — Semantics (Formulas & Programs)

Semantics of PDL is interpreted over *Labeled Transition Systems* $M = (W, R, V)$.

- W : non-empty set of worlds/states
- R : mapping from $\mathcal{P}$ (atomic programs) to W^2 (binary relations over W)
- V : mapping $\mathcal{A}$ (atomic formulas) to subsets of W (truth-sets)

Complex programs: interpreted as composite binary relations over W

$\|\alpha\|$, the **interpretation of a program** $\alpha \in \mathbf{P}$, defined inductively ($\delta \in \mathcal{P}; \beta, \gamma \in \mathbf{P}; A \in \mathbf{F}$):

$$\begin{aligned}\|\delta\| &\equiv R(\delta) \in W^2 \\ \|\beta; \gamma\| &\equiv \{(w, v) \in W^2 \mid \exists u \in W. (w, u) \in \|\beta\| \text{ and } (u, v) \in \|\gamma\|\} \\ \|\beta \cup \gamma\| &\equiv \|\beta\| \cup \|\gamma\| \\ \|\beta^*\| &\equiv (\|\beta\|)^* && R^* \text{ is reflexive-transitive closure of } R \\ \|A?\| &\equiv \{(w, w) \in W^2 \mid M, w \models A\}\end{aligned}$$

$M, w \models F$, the **evaluation of formula** F (in M, w), is defined as ($A \in \mathcal{A}; B, C \in \mathbf{F}; \alpha \in \mathbf{P}$):

$$\begin{aligned}M, w \models A &\quad \text{iff} \quad w \in V(A) \\ M, w \models \neg B &\quad \text{iff} \quad \text{not } M, w \models B \\ M, w \models B \vee C &\quad \text{iff} \quad M, w \models B \text{ or } M, w \models C \\ M, w \models [\alpha]B &\quad \text{iff} \quad M, v \models B \text{ for every } v \in W \text{ s.t. } (w, v) \in \|\alpha\|\end{aligned}$$

F is **valid** in M ($M \models F$), whenever $M, w \models F$ for every $w \in W$.

F is **valid** ($\models F$), if $M \models F$ for every M

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

$M, w \models A$	iff	$w \in V(A)$
$M, w \models \neg B$	iff	not $M, w \models B$
$M, w \models B \vee C$	iff	$M, w \models B$ or $M, w \models C$
$M, w \models [\alpha]B$	iff	$M, v \models B$ for every $v \in W$ s.t. $(w, v) \in \ \alpha\ $

(**PDL Formulas (semantically embedded in HOL)**)

```
definition pdlfalse::stSet ("⊥")                where "⊥ ≡ λw. False"
definition pdlnot::"stSet⇒stSet" ("¬" [52]53)    where "¬φ ≡ λw. ¬(φ w)"
definition pdlor::"stSet⇒stSet⇒stSet" (infixr"∨"50) where "φ∨ψ ≡ λw. φ w ∨ ψ w"
definition pdlbox::"pr⇒stSet⇒stSet" ("[_]" [52,52]53) where "[p]φ ≡ λw. ∀v. p w v ⟶ φ v"

(*validity of PDL formulas: "grounding" of PDL formulas in meta-logic HOL*)
definition valid::"stSet⇒bool" ("⊨" [8]109)      where "⊨φ ≡ ∀w. φ w"
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

$M, w \models A$	iff	$w \in V(A)$
$M, w \models \neg B$	iff	$\text{not } M, w \models B$
$M, w \models B \vee C$	iff	$M, w \models B \text{ or } M, w \models C$
$M, w \models [\alpha]B$	iff	$M, v \models B \text{ for every } v \in W \text{ s.t. } (w, v) \in \ \alpha\ $

(**PDL Formulas (semantically embedded in HOL)**)

```
definition pdlfalse::stSet ("⊥")                where "⊥ ≡ λw. False"
definition pdlnot::"stSet⇒stSet" ("¬" [52]53)    where "¬φ ≡ λw. ¬(φ w)"
definition pdlor::"stSet⇒stSet⇒stSet" (infixr"∨"50) where "φ∨ψ ≡ λw. φ w ∨ ψ w"
definition pdlbox::"pr⇒stSet⇒stSet" ("[_]" [52,52]53) where "[p]φ ≡ λw. ∀v. p w v ⟶ φ v"

(*validity of PDL formulas: "grounding" of PDL formulas in meta-logic HOL*)
definition valid::"stSet⇒bool" ("⊨" [8]109)      where "⊨φ ≡ ∀w. φ w"
```

```
(*Some basic types*)
datatype var = x | y | z    (*variables*)
type_synonym val = nat     (*values*)

type_synonym st = "var⇒val"    (*states*)
type_synonym pr = "st⇒st⇒bool" (*programs*)
type_synonym stSet = "st⇒bool" (*sets/properties of states*)
type_synonym prSet = "pr⇒bool" (*sets/properties of programs*)
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

$$\begin{aligned} \|\delta\| &\equiv R(\delta) \in W^2 \\ \|\beta;\gamma\| &\equiv \{(w, v) \in W^2 \mid \exists u \in W. (w, u) \in \|\beta\| \text{ and } (u, v) \in \|\gamma\|\} \\ \|\beta \cup \gamma\| &\equiv \|\beta\| \cup \|\gamma\| \\ \|\beta^*\| &\equiv (\|\beta\|)^* \\ \|A?\| &\equiv \{(w, w) \in W^2 \mid M, w \models A\} \end{aligned}$$

R^* is reflexive-transitive closure of R

(**PDL Programs (semantically embedded in HOL); extended for real assignment**)

```
definition pdlSeqComp::"pr⇒pr⇒pr" (infixr";"60) where "r;s ≡ λw v. ∃u. r w u ∧ s u v"
definition pdlUnion::"pr⇒pr⇒pr" (infixr"U"60) where "rUs ≡ λw v. r w v ∨ s w v"
definition pdlIt::"pr⇒pr" ("_*"61) where "r* ≡ transClos (reflClos r)"
definition pdlTest::"stSet⇒pr" ("_?"65) where "φ? ≡ λw v. w = v ∧ φ v"
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

$$\begin{aligned} \|\delta\| &\equiv R(\delta) \in W^2 \\ \|\beta;\gamma\| &\equiv \{(w, v) \in W^2 \mid \exists u \in W. (w, u) \in \|\beta\| \text{ and } (u, v) \in \|\gamma\|\} \\ \|\beta \cup \gamma\| &\equiv \|\beta\| \cup \|\gamma\| \\ \|\beta^*\| &\equiv (\|\beta\|)^* \\ \|\mathbf{A?}\| &\equiv \{(w, w) \in W^2 \mid M, w \models A\} \end{aligned}$$

R^* is reflexive-transitive closure of R

(**PDL Programs (semantically embedded in HOL); extended for real assignment**)

```
definition pdlSeqComp::"pr⇒pr⇒pr" (infixr";"60) where "r;s ≡ λw v. ∃u. r w u ∧ s u v"
definition pdlUnion::"pr⇒pr⇒pr" (infixr"U"60) where "rUs ≡ λw v. r w v ∨ s w v"
definition pdlIt::"pr⇒pr" ("_*"61) where "r* ≡ transClos (reflClos r)"
definition pdlTest::"stSet⇒pr" ("_?"65) where "φ? ≡ λw v. w = v ∧ φ v"
```

(*Properties on programs, which are relations on states*)

```
definition refl::prSet where "refl r ≡ ∀w. r w w"
definition trans::prSet where "trans r ≡ ∀v.∀w.∀u. (r v w ∧ r w u) → r v u"
definition relSubsetEq::"pr⇒pr⇒bool" where "relSubsetEq r s ≡ ∀v.∀w. r v w → s v w"
definition reflClos::"pr⇒pr" where "reflClos r ≡ λv. λw. v = w ∨ r v w"
definition transClos::"pr⇒pr" where
  "transClos r ≡ λv.λw.∀q. trans q → ((relSubsetEq r q) → q v w)"
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

(**PDL Formulas (semantically embedded in HOL)**)

```
definition pdlfalse::stSet ("⊥") where "⊥ ≡ λw. False"
definition pdlnot::"stSet⇒stSet" ("¬" [52]53) where "¬φ ≡ λw. ¬(φ w)"
definition pdlor::"stSet⇒stSet⇒stSet" (infixr"∨"50) where "φ∨ψ ≡ λw. φ w ∨ ψ w"
definition pdlbox::"pr⇒stSet⇒stSet" ("□" [52,52]53) where "[p]φ ≡ λw. ∀v. p w v ⟶ φ v"

(*validity of PDL formulas: "grounding" of PDL formulas in meta-logic HOL*)
definition valid::"stSet⇒bool" ("□" [8]109) where "□φ ≡ ∀w. φ w"
```

(**PDL Programs (semantically embedded in HOL); extended for real assignment**)

```
definition pdlSeqComp::"pr⇒pr⇒pr" (infixr";"60) where "r;s ≡ λw v. ∃u. r w u ∧ s u v"
definition pdlUnion::"pr⇒pr⇒pr" (infixr"∪"60) where "r∪s ≡ λw v. r w v ∨ s w v"
definition pdlIt::"pr⇒pr" ("*"61) where "r* ≡ transClos (reflClos r)"
definition pdlTest::"stSet⇒pr" ("_"?65) where "φ? ≡ λw v. w = v ∧ φ v"
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

```
(**Define some intuitive syntactic sugar for programs and for properties**)
abbreviation pdlCond ("if _ then _ else _"70) where "if  $\varphi$  then r else s  $\equiv$  ( $\varphi$ ?;r)  $\cup$  (( $\neg\varphi$ )?;s)"
abbreviation pdlWhile ("while _ do _"70) where "while  $\varphi$  do r  $\equiv$  ( $\varphi$ ?;r)*;(( $\neg\varphi$ )?)"
abbreviation pdlSkip ("skip") where "skip  $\equiv$  T?"

(*example: if x is e, then after executing if(x=e,x:=f,x:=g) x is of value f*)
lemma "[ (x =? e)  $\rightarrow$  [if (x =? e) then (x := f) else (x := g)](x =? f) ]"
  unfolding pdlProgs unfolding pdlInHOL by auto
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

```
(**Define some intuitive syntactic sugar for programs and for properties**)
abbreviation pdlCond ("if _ then _ else _"?) where "if  $\varphi$  then r else s  $\equiv$  ( $\varphi$ ?;r)  $\cup$  (( $\neg\varphi$ )?;s)"
abbreviation pdlWhile ("while _ do _"?) where "while  $\varphi$  do r  $\equiv$  ( $\varphi$ ?;r)*;(( $\neg\varphi$ )?)"
abbreviation pdlSkip ("skip") where "skip  $\equiv$  T?"

(*example: if x is e, then after executing if(x=e,x:=f,x:=g) x is of value f*)
lemma "[ $\{x = ? e\} \rightarrow [\text{if } (x = ? e) \text{ then } (x := f) \text{ else } (x := g)](x = ? f)$ ]"
  unfolding pdlProgs unfolding pdlInHOL by auto
```

```
(**Hoare-style triples can be defined in terms of PDL**)
abbreviation hoareTriple ("{ _ } { _ }") where "{ $\varphi$ } P { $\psi$ }  $\equiv$  [ $\varphi \rightarrow ([P]\psi)$ ]"

(*Now the Hoare axioms can be derived(!) as theorems of the system.*)
lemma ruleSkip: "{ $\varphi$ } skip { $\varphi$ }" unfolding pdlProgs unfolding pdlInHOL by auto
lemma ruleComp: "{ $\varphi$ } P { $\psi$ }  $\Rightarrow$  { $\psi$ } Q { $\xi$ }  $\Rightarrow$  { $\varphi$ } P;Q { $\xi$ }" unfolding pdlProgs pdlInHOL by auto
lemma ruleIfthe: "{ $\varphi \wedge \psi$ } S { $\xi$ }  $\Rightarrow$  { $\neg\varphi \wedge \psi$ } T { $\xi$ }  $\Rightarrow$  { $\psi$ } if  $\varphi$  then S else T { $\xi$ }"
  unfolding pdlProgs unfolding pdlInHOL by auto
lemma ruleConseq1: "{ $\varphi$ } P { $\psi$ }  $\Rightarrow$  [ $\psi \rightarrow \xi$ ]  $\Rightarrow$  { $\varphi$ } P { $\xi$ }" unfolding pdlProgs pdlInHOL by auto
lemma ruleConseq2: "[ $\xi \rightarrow \varphi$ ]  $\Rightarrow$  { $\varphi$ } P { $\psi$ }  $\Rightarrow$  { $\xi$ } P { $\psi$ }" unfolding pdlProgs pdlInHOL by auto
lemma ruleWhile: "{ $\varphi \wedge \psi$ } P { $\varphi$ }  $\Rightarrow$  { $\varphi$ } (while  $\psi$  do P) { $\neg\psi \wedge \varphi$ }" sorry (*proof omitted here*)
```

DEMO in Isabelle/HOL: Interactive-Automated Program Reasoning

```
(**Define some intuitive syntactic sugar for programs and for properties**)
abbreviation pdlCond ("if _ then _ else _?") where "if  $\varphi$  then r else s  $\equiv (\varphi?;r) \cup ((\neg\varphi)?;s)"$ 
abbreviation pdlWhile ("while _ do _?") where "while  $\varphi$  do r  $\equiv (\varphi?;r)^* ; ((\neg\varphi)?)"$ 
abbreviation pdlSkip ("skip") where "skip  $\equiv T?"$ 

(*example: if x is e, then after executing if(x=e,x:=f,x:=g) x is of value f*)
lemma "[ $(x = ? e) \rightarrow [if (x = ? e) then (x := f) else (x := g)](x = ? f)$ ]"
  unfolding pdlProgs unfolding pdlInHOL by auto
```

```
(**Hoare-style triples can be defined in terms of PDL**)
abbreviation hoareTriple ("{ $\varphi$ }. { $\psi$ }") where "{ $\varphi$ } P { $\psi$ }  $\equiv [\varphi \rightarrow ([P]\psi)]$ "

(*Now the Hoare axioms can be derived(!) as theorems of the system.*)
lemma ruleSkip: "{ $\varphi$ } skip { $\varphi$ }" unfolding pdlProgs unfolding pdlInHOL by auto
lemma ruleComp: "{ $\varphi$ } P { $\psi$ }  $\Rightarrow \{\psi\} Q \{\xi\} \Rightarrow \{\varphi\} P;Q \{\xi\}"$  unfolding pdlProgs pdlInHOL by auto
lemma ruleIfthe: "{ $\varphi \wedge \psi$ } S { $\xi$ }  $\Rightarrow \{\neg\varphi \wedge \psi\} T \{\xi\} \Rightarrow \{\psi\} if \varphi then S else T \{\xi\}"$ 
  unfolding pdlProgs unfolding pdlInHOL by auto
lemma ruleConseq1: "{ $\varphi$ } P { $\psi$ }  $\Rightarrow [\psi \rightarrow \xi] \Rightarrow \{\varphi\} P \{\xi\}"$  unfolding pdlProgs pdlInHOL by auto
lemma ruleConseq2: "[ $\xi \rightarrow \varphi$ ]  $\Rightarrow \{\varphi\} P \{\psi\} \Rightarrow \{\xi\} P \{\psi\}"$  unfolding pdlProgs pdlInHOL by auto
lemma ruleWhile: "{ $\varphi \wedge \psi$ } P { $\varphi$ }  $\Rightarrow \{\varphi\} (while \psi do P) \{\neg\psi \wedge \varphi\}"$  sorry (*proof omitted here*)
```

```
(*we can now reformulate earlier experiments so that they look nicer*)
lemma "{ $x = ? 0 \wedge y = ? 0$ } (x := 1);(y := 1) { $x == ? y$ }"
  unfolding pdlProgs pdlInHOL by auto
lemma "{ $x = ? 0 \wedge y = ? 0$ } (x := 1);(y := 0) { $x == ? y$ }"
  unfolding pdlProgs pdlInHOL nitpick oops (*countermodel*)
lemma "{ $x = ? e$ } {if (x = ? e) then (x := f) else (x := g)} { $x = ? f$ }"
  unfolding pdlProgs pdlInHOL by auto
lemma "{ $x = ? 2$ } {if (x = ? 2) then (x := 3) else (x := 5)} { $x = ? 3$ }"
  unfolding pdlProgs pdlInHOL by auto
lemma "{ $x = ? 4$ } {if (x = ? 2) then (x := 3) else (x := 5)} { $x = ? 3$ }"
  unfolding pdlProgs pdlInHOL nitpick oops (*countermodel*)
lemma "{ $x = ? 0 \wedge y = ? 0$ } (x := 1);(y := 2);(if (x == ? y) then (x := 1) else (x := 0)) { $x = ? 0$ }"
  unfolding pdlProgs pdlInHOL by auto
```

Reasoning Systems and Tools

- ▶ ATP: HOL, FOL, PL
- ▶ ITP and ITP-ATP integration, “hammer”-architectures
- ▶ user-interaction, proof planning, agent-based reasoning
- ▶ proof representation and proof data-structures

Logics and Theory

- ▶ HOL: semantics and proof theory
- ▶ wide range of (expressive) non-classical logics
- ▶ universal meta-logical reasoning via shallow semantical embeddings in HOL

Different possible further directions

- ▶ traditional software and hardware verification
- ▶ verification of restricted contract languages
- ▶ legal & ethical compliance of intelligent autonomous agents

Many other supervision and lecturing competencies

— end of first presentation —

Reasoning about Programs, Agents and Machine Ethics — A Meta-logical Approach —

Christoph Benzmüller

Freie Universität Berlin | University of Luxembourg



TU Vienna — October 22, 2019

Reasoning about Programs, Agents and Machine Ethics — A Meta-logical Approach —

Christoph Benzmüller

Freie Universität Berlin | University of Luxembourg



TU Vienna — October 22, 2019

Reasoning about Programs, Agents and Machine Ethics — A Meta-logical Approach —

Christoph Benzmüller

Freie Universität Berlin | University of Luxembourg



TU Vienna — October 22, 2019

- ▶ Part I: Progress in Interactive-Automated Reasoning in HOL
 - ▶ Leo-family of ATP systems for HOL *since 1997*
 - ▶ HOL-ATP infrastructure TPTP THF *since 2007*
 - ▶ semantics and proof theory of HOL *since 1996*

- ▶ Part II: Universal Meta-Logical Reasoning
 - ▶ shallow semantical embeddings of expressive non-classical logics in HOL *since 2008*
 - ▶ interactive-automated reasoning with expressive non-classical logics *since 2012*
 - ▶ applications beyond program reasoning: philosophy, mathematics, NL *since 2013*

- ▶ Part III: Thrustworthy & Explainable AI, Machine Ethics
 - ▶ ethical governor architectures *since 2018*
 - ▶ flexible normative reasoning and deontic logics *since 2017*
 - ▶ mechanization/automation of ethical theories *since 2018*

Recent successes in (GOF)AI

- ▶ Computer-verification of Kepler's conjecture
- ▶ Automated solution of open mathematical problems
- ▶ Automation of arguments in metaphysics (with LEO-II)



Symbolic AI (here Theorem Proving)

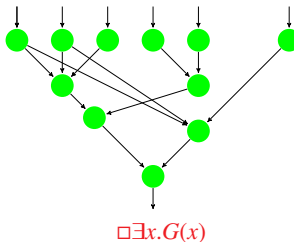
Conjecture (and Premises)

$\Box \exists x. G(x)$



$$\frac{A \quad A \Rightarrow B}{B} \text{ modus ponens}$$

Axioms & Rules of Logic



Recent successes in (GOF)AI

- ▶ Computer-verification of Kepler's conjecture
- ▶ Automated solution of open mathematical problems
- ▶ Automation of arguments in metaphysics (with LEO-II)



Symbolic AI (here Theorem Proving)

Conjecture (ar
 $\Box \exists x. G(x)$)



C. Benzmüller, 2019

Home | Video | Themen | Forum | English | DER SPIEGEL | SPIEGEL TV | Abo | Shop

RSS | Mobile | Newsletter

SPIEGEL ONLINE INTERNATIONAL

Sign in | Register

Front Page World Europe Germany Business Zeitgeist Newsletter

English Site > Germany > Science > Scientists Use Computer to Mathematically Prove Gödel's God Theorem

Holy Logic: Computer Scientists 'Prove' God Exists

By David Knight



— Part I —

Progress in Interactive-Automated Reasoning in HOL



$\forall$ P.Leo III

with A. Steen, M. Wisniewski

What is Leo-III?

- ▶ ATP for HOL based on polymorphic(!) extensional higher-order paramodulation
- ▶ integrates SMT- & FO-ATPs
- ▶ supports **all TPTP syntax dialects**
- ▶ native support for **more than 120 modal logics** (normal quantified modal logics)
- ▶ including native support for **quantified deontic logics**
- ▶ Website: <http://page.mi.fu-berlin.de/lex/leo3/>

Recent Independent Evaluation:

- ▶ (Brown et al) GRUNGE: A Grand Unified ATP Challenge, CADE-27
- ▶ Problems are stemming from the HOL4 proof library — **Program Reasoning!**



with A. Steen, M. Wisniewski

What is Leo-III?

- ▶ ATP for HOL based on polymorphic(!) extensional higher-order paramodulation
- ▶ integrates SMT- & FO-ATPs
- ▶ supports **all TPTP syntax dialects**
- ▶ native support for **more than 120 modal logics** (normal quantified modal logics)
- ▶ including native support for **quantified deontic logics**
- ▶ Website: <http://page.mi.fu-berlin.de/lex/leo3/>

Recent Independent Evaluation:

- ▶ (Brown et al) GRUNGE: A Grand Unified ATP Challenge, CADE-27
- ▶ Problems are stemming from the HOL4 proof library — **Program Reasoning!**



with A. Steen, M. Wisniewski

What is Leo-III?

- ▶ ATP for HOL based on polymorphic(!) extensional higher-order paramodulation
- ▶ integrates SMT- & FO-ATPs
- ▶ supports **all TPTP syntax dialects**
- ▶ native support for **more than 120 modal logics** (normal quantified modal logics)
- ▶ including native support for **quantified deontic logics**
- ▶ Website: <http://page.mi.fu-berlin.de/lex/leo3/>

Recent Independent Evaluation:

- ▶ (Brown et al) GRUNGE: A Grand Unified ATP Challenge, CADE-27
- ▶ Problems are stemming from the HOL4 proof library — **Program Reasoning!**

GRUNGE: A Grand Unified ATP Challenge

Chad E. Brown, Thibault Gauthier, Cezary Kaliszyk, Geoff Sutcliffe, Josef Urban

(Submitted on 6 Mar 2019)

This paper describes a large set of related theorem proving problems obtained by translating theorems from the HOL4 standard library into multiple logical formalisms. The formalisms are in higher-order logic (with and without type variables) and first-order logic (possibly with multiple types, and possibly with type variables). The resultant problem sets allow us to run automated theorem provers that support different logical formats on corresponding problems, and compare their performances. This also results in a new "grand unified" large theory benchmark that emulates the ITP/ATP hammer setting, where systems and metasytems can use multiple ATP formalisms in complementary ways, and jointly learn from the accumulated knowledge.

System	TH1-I	TH0-I	TH0-II	TF1-I	TF0-I	TF0-II	FOF-I	FOF-II	Union
agsyHOL		1374	1187						1605
Beagle					2007	2047			2531
cocATP		899	599						1000
CSE_E							4251	3102	4480
CVC4					4851	3991			5252
E					4277	3622	4618	3844	5118
HOLyHammer	5059								5059
iProver							2778	2894	3355
iProverMo'					2435	1639			2699
LEO-II		2579	1923						3213
Leo-III	6668	5018	3485	3458	4032	3421			7062
Metis							2353	474	2356
Princess					3646	2138			3849
Prover9							2894	1742	3128
Satallax		2207	1292						2494
SPASS							2850	3349	3821
Vampire					4837	4693	4008	4928	5929
Zipperp'n		2252	2161	3771	3099	2576			4203
Union	6824	5209	3771	4608	5732	5073	5165	5108	7377

Table 2. Number of theorems proved, out of 12140.

**Leo III**

Leo III - A MASSIVELY PARALLEL HIGHER-ORDER THEOREM PROVER

Overview**People****Publications****Theses**

News

Leo-III wins the CASC LTB division!

At the most recent CASC-27, Leo-III 1.4 ranked 1st place in the practically relevant Large Theory Batch (LTB) division. With over 5400 of 1000 solved problems (54%), Leo-III completes the challenge with approx. 800 problems lead before the next ATP systems, E and Vampire. Also, Leo-III came in second after Satallax, in the THF division of CASC.

New Study on Leo-III's effectivity

An independent, large evaluation study identifies Leo-III as the worlds leading and most general higher-order automated theorem prover. For further details, see the pre-print of [GRUNGE: A Grand Unified ATP Challenge](#) (Chad E. Brown, Thibault Gauthier, Cezary Kaliszyk, Geoff Sutcliffe, Josef Urban) on [arXiv.org](#).



∇**P**.Leo III

Freie Universitt



Leo III - A MASSIVELY PARALLEL HIGHER-ORDER THEOREM PROVER

Overview

People

Publications

These

News

Leo-III wins the CASC LTB division!

At the most recent CASC-27, Leo-III 1.4 ranked 1st place in the practically relevant Large Theory Batch (LTB) division. With over 5400 of 1000 solved problems (54%), Leo-III completes the challenge with approx. 800 problems lead before the next ATP systems, E and Vampire. Also, Leo-III came in second after Satallax, in the THF division of CASC.

New Study on Leo-III's effectivity

An independent, large evaluation study identifies Leo-III as the worlds leading and most general higher-order automated theorem prover. For further details, see the pre-print of [GRUNGE: A Grand Unified ATP Challenge](#) (Chad E. Brown, Thibault Gauthier, Cezary Kaliszyk, Geoff Sutcliffe, Josef Urban) on arXiv.org.

Large Theory Batch Problems	Leo-III LTB-1.4	E LTB-2.4	MaLARE 0.8	Vampire LTB-4.4	MaLARE 0.6	iProver LTB-3.0	GKC LTB-0.4
Solved/10000	5440/10000	4644/10000	4640/10000	4499/10000	4380/10000	3793/10000	2755/10000
Solutions	5440 54%	4644 46%	4640 46%	4499 44%	4380 43%	3789 37%	2755 27%

**CORDIS**
EU research results

English 

Search

 Sign in 

An Infrastructure for Typed Higher-order Form Automated Theorem Proving

[Fact Sheet](#) [Result in Brief](#) [Reporting](#) [Results](#)

Final Report Summary - THFTPTP (An infrastructure for typed higher-order form automated theorem proving)

There is a well established infrastructure that supports research, development, and deployment of first-order Automated Theorem Proving (ATP) systems, stemming from the Thousands of Problems for Theorem Provers (TPTP) problem library. This infrastructure includes the TPTP itself, the TPTP language and SZS result ontology, the Thousands of Solutions from Theorem Provers (TSTP) solution library, various tools associated with the libraries, and the CADE ATP System Competition (CASC). This infrastructure has been central to the impressive progress that has been made in the development of high performance first-order ATP systems.

Summary of accomplishments

Project information

THFTPTP

Grant agreement ID: 219982

Status
Closed project

Start date	End date
15 July 2008	14 September 2009

Funded under:
FP7-PEOPLE

Overall budget:
€ 130 109,59



Journal of Formalized Reasoning

HOME ABOUT LOGIN REGISTER SEARCH CURRENT ARCHIVES ANNOUNCEMENTS

Home > Vol 3, No 1 (2010) > Sutcliffe

Automated Reasoning in Higher-Order Logic using the TPTP THF Infrastructure

Geoff Sutcliffe, Christoph Benzmueller

Abstract

The THF infrastructure of the HOL-ATP application is higher order logic and the use of

```
% Comments : THF0 syntax
%-----
%---Basic set theory definitions
include('Axioms/SET008~0.ax').
%-----
thf(union_distributes_over_intersection,conjecture,(
  ! [A: $i > $o,B: $i > $o,C: $i > $o] :
    ( ( union @ A @ ( intersection @ B @ C ) )
      = ( intersection @ ( union @ A @ B ) @ ( union @ A @ C ) ) ) ).
%-----
```

Fig. 1. The TPTP problem file SET171~3.p

The Journal of Symbolic Logic



Article

Metrics

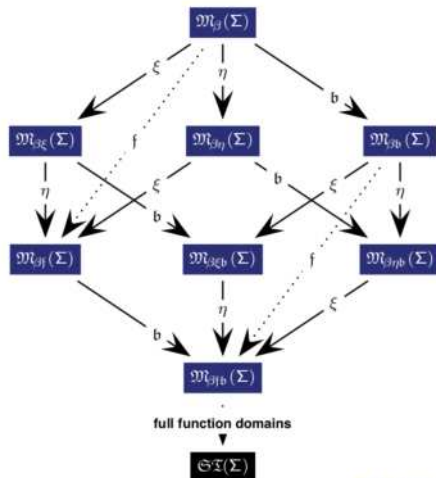
Volume 69, Issue 4 December 2004, pp. 1027-1088

Higher-order semantics and extensionalityChristoph Benzmüller ^(a1), Chad E. Brown ^(a2) and Michael Kohlhase ^(a3) ^(a4) DOI: <https://doi.org/10.2178/jsl/1102022211> Published online by Cambridge University Press: 12 March 2014**Abstract.**

In this paper we re-examine the semantics of classical higher-order logic with the purpose of clarifying the role of extensionality. To reach this goal, we distinguish nine classes of higher-order models with respect to various combinations of Boolean extensionality and three forms of functional extensionality. Furthermore, we develop a methodology of abstract consistency methods (by providing the necessary model existence theorems) needed to analyze completeness of (machine-oriented) higher-order calculi with respect to these model classes.



Semantics: HOL-CUBE



elementary type theory (Andrews)

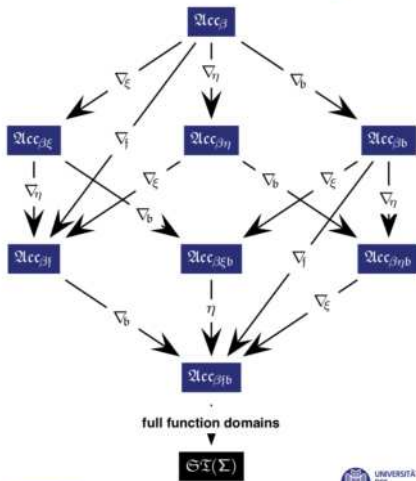
see [Journal of Symbolic Logic (2004) 69(4)]

extensional type theory (Henkin semantics)

standard semantics



Abstract Consistency



Properties for Acc_β

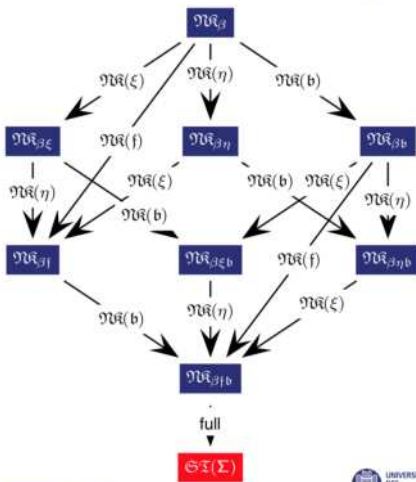
∇_ϵ	...	$\nabla_\vee$	...
$\nabla_\neg$	...	$\nabla_\wedge$	...
∇_β	...	$\nabla_\forall$	...
		$\nabla_\exists$	...

Properties for Extensionality

- ∇_b If $\neg(A \doteq^\alpha B) \in \Phi$, then $\Phi, A, \neg B \in \mathbb{F}_\Sigma$ or $\Phi, \neg A, B \in \mathbb{F}_\Sigma$.
- ∇_η If $A \doteq^\eta B$ and $A \in \Phi$, then $\Phi, B \in \mathbb{F}_\Sigma$.
- ∇_ϵ If $\neg(\lambda X_\alpha. M \doteq^{\alpha \rightarrow \beta} \lambda X_\alpha. N) \in \Phi$, then $\Phi, \neg([w/X]M \doteq^\beta [w/X]N) \in \mathbb{F}_\Sigma$ for any new $w_\alpha \in \Sigma_\alpha$.
- ∇_η If $\neg(G \doteq^{\alpha \rightarrow \beta} H) \in \Phi$, then $\Phi, \neg(Gw \doteq^\beta Hw) \in \mathbb{F}_\Sigma$ for any new $w_\alpha \in \Sigma_\alpha$.



ND for HOL: The Complete Picture



Base Calculus $\mathcal{N}\mathcal{R}_\beta$

- $\mathcal{N}\mathcal{R}(Hyp)$ — $\mathcal{N}\mathcal{R}(\beta)$
- $\mathcal{N}\mathcal{R}(\neg I)$ — $\mathcal{N}\mathcal{R}(\neg E)$
- $\mathcal{N}\mathcal{R}(\vee I_L)$ — $\mathcal{N}\mathcal{R}(\vee I_R)$
- $\mathcal{N}\mathcal{R}(\vee E)$
- $\mathcal{N}\mathcal{R}(\Pi I)^w$
- $\mathcal{N}\mathcal{R}(\Pi E)$ — $\mathcal{N}\mathcal{R}(Contr)$

Optional Extensionality Rules

- $\mathcal{N}\mathcal{R}(\eta)$ — $\mathcal{N}\mathcal{R}(\xi)$
- $\mathcal{N}\mathcal{R}(f)$ — $\mathcal{N}\mathcal{R}(b)$

LOGICAL METHODS
IN COMPUTER SCIENCEBenzmueller, Christoph and Brown, Chad E. and Kohlhase, Michael - **Cut-Simulation and Impredicativity** ^

lmcs:1144 - Logical Methods in Computer Science, March 3, 2009, Volume 5, Issue 1

Cut-Simulation and Impredicativity*Authors: Benzmueller, Christoph and Brown, Chad E. and Kohlhase, Michael*

We investigate cut-elimination and cut-simulation in impredicative (higher-order) logics. We illustrate that adding simple axioms such as Leibniz equations to a calculus for an impredicative logic -- in our case a sequent calculus for classical type theory -- is like adding cut. The phenomenon equally applies to prominent axioms like Boolean- and functional extensionality, induction, choice, and description. This calls for the development of calculi where these principles are built-in instead of being treated axiomatically.

Benzmueller, Christof
lmcs:1144 - Logical M

Cut-Simulation and

Authors: Benzmueller

We investigate cut-elimination
Leibniz equations to a certain
The phenomenon equally
This calls for the development

Definition 1 (Sequent calculus $\mathcal{G}_{\beta\text{fb}}$) Let Δ and Δ' be finite sets of β -normal closed formulas of HOL and let Δ, s denote the set $\Delta \cup \{s\}$. The sequent calculus $\mathcal{G}_{\beta\text{fb}}$ comprises the following rules:

$$\text{Basic Rules} \quad \frac{\Delta, s}{\Delta, \neg s} \mathcal{G}(\neg) \quad \frac{\Delta, \neg s \quad \Delta, \neg t}{\Delta, \neg(s \vee t)} \mathcal{G}(\vee_-) \quad \frac{\Delta, s, t}{\Delta, (s \vee t)} \mathcal{G}(\vee_+)$$

$$\frac{\Delta, \neg(sl) \downarrow_{\beta} \quad l_{\alpha} \text{ closed term}}{\Delta, \neg \Pi^{\alpha} s} \mathcal{G}(\Pi_-^l) \quad \frac{\Delta, (sc) \downarrow_{\beta} \quad c_{\delta} \text{ new symbol}}{\Delta, \Pi^{\alpha} s} \mathcal{G}(\Pi_+^c)$$

$$\text{Initialization} \quad \frac{s \text{ atomic (and } \beta\text{-normal)}}{\Delta, s, \neg s} \mathcal{G}(\text{init})$$

$$\frac{\Delta, (s \doteq^o t) \quad s, t \text{ atomic}}{\Delta, \neg s, t} \mathcal{G}(\text{Init}^{\doteq})$$

$$\text{Extensionality} \quad \frac{\Delta, (\forall X_{\alpha} s X \doteq^{\beta} t X) \downarrow_{\beta}}{\Delta, (s \doteq^{\alpha \rightarrow \beta} t)} \mathcal{G}(\text{f}) \quad \frac{\Delta, \neg s, t \quad \Delta, \neg t, s}{\Delta, (s \doteq^o t)} \mathcal{G}(\text{b})$$

$$\text{Decomposition} \quad \frac{\Delta, (s^1 \doteq^{\alpha_1} t^1) \quad \dots \quad \Delta, (s^n \doteq^{\alpha_n} t^n) \quad n \geq 1, \beta \in \{o, i\}, \quad h_{\alpha^n \rightarrow \beta} \in \Sigma}{\Delta, (hs^n \doteq^{\beta} ht^n)} \mathcal{G}(\text{d})$$

Benzmueller, Christ
lmcs:1144 - Logical M

Cut-Simulation and

Authors: Benzmueller

We investigate cut-elimin
Leibniz equations to a ca
The phenomenon equal
This calls for the develop

Definition 1 (Sequent calculus $\mathcal{G}_{\beta\text{fb}}$) Let Δ and Δ' be finite sets of β -normal closed formulas of HOL and let Δ, s denote the set $\Delta \cup \{s\}$. The sequent calculus $\mathcal{G}_{\beta\text{fb}}$ comprises the following rules:

Basic Rules

$$\frac{\Delta, s}{\Delta, \neg\neg s} \mathcal{G}(\neg) \quad \frac{\Delta, \neg s \quad \Delta, \neg t}{\Delta, \neg(s \vee t)} \mathcal{G}(\vee-) \quad \frac{\Delta, s, t}{\Delta, (s \vee t)} \mathcal{G}(\vee+)$$

$$\frac{\Delta, \neg(sl) \downarrow_{\beta} \quad l_{\alpha} \text{ closed term}}{\Delta, \neg\Pi^{\alpha} s} \mathcal{G}(\Pi_-) \quad \frac{\Delta, (sc) \downarrow_{\beta} \quad c_{\delta} \text{ new symbol}}{\Delta, \Pi^{\alpha} s} \mathcal{G}(\Pi_+^c)$$

Initialization

$$\frac{s \text{ atomic (and } \beta\text{-normal)}}{\Delta, s \multimap s} \mathcal{G}(\text{init})$$

[Journal of Philosophical Logic](#)

June 2017, Volume 46, [Issue 3](#), pp 333–353 | [Cite as](#)

Cut-Elimination for Quantified Conditional Logic

Authors

[Authors and affiliations](#)

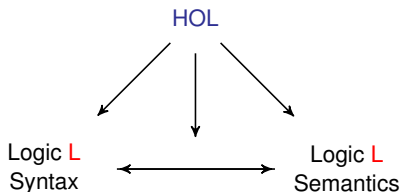
Christoph Benzmüller 

Decomposition

$$\frac{\Delta, (hs^n \doteq^{\beta} ht^n)}{\Delta, (hs^n \doteq^{\beta} ht^n)} \mathcal{G}(d)$$

— Part II —

Universal Meta-Logical Reasoning



Examples for **L** we have already studied:

Intuitionistic Logics, (Mathematical) Fuzzy Logics, Free Logic, **Multimodal Logics**, Description Logics, Conditional Logics, Access Control Logics, Hybrid Logics, Multivalued Logics, Deontic Logics, Paraconsistent Logics, Computation Tree Logic, **Dynamic Logic**,...

Embedding works also for quantifiers (first-order & higher-order)

HOL provers become universal logic reasoning engines!

interactive: Isabelle/HOL, PVS, HOL4, Hol Light, Coq/HOL, ...

automated: Leo-III, LEO-II, Satallax, TPS, Nitpick, Isabelle/HOL, ...



STUDIES IN LOGIC
AND
PRACTICAL REASONING

VOLUME 3

D.M. GABBAY / P. GARDENFORS / J. SIEKMANN / J. VAN BENTHEM / M. VARDI / J. WOODS

EDITORS

*Handbook of
Modal Logic*

2 BASIC MODAL LOGIC

In this section we introduce the basic modal language and its relational semantics. We define basic modal syntax, introduce models and frames, and give the satisfaction definition. We then draw the reader's attention to the internal perspective that modal languages offer on relational structure, and explain why models and frames should be thought of as graphs. Following this we give the standard translation. This enables us to convert any basic modal formula into a first-order formula with one free variable. The standard translation is a bridge between the modal and classical worlds, a bridge that underlies much of the work of this chapter.

2.1 First steps in relational semantics

Suppose we have a set of proposition symbols (whose elements we typically write as p, q, r and so on) and a set of modality symbols (whose elements we typically write as m, m', m'' , and so on). The choice of PROP and MOD is called the *signature* (or *similarity type*) of the language; in what follows we'll tacitly assume that PROP is denumerably infinite, and we'll often work with signatures in which MOD contains only a single element. Given a signature, we define the *basic modal language* (over the signature) as follows:

$$\varphi ::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \mid \langle m \rangle\varphi \mid [m]\varphi.$$

That is, a basic modal formula is either a proposition symbol, a boolean constant, a boolean combination of basic modal formulas, or (most interesting of all) a formula prefixed by a diamond

2 BASIC MODAL LOGIC

In this section we introduce the basic modal language and its relational semantics. We define basic modal syntax, introduce models and frames, and give the satisfaction definition. We then draw the reader's attention to the internal perspective that modal languages offer on relational structure, and explain why models and frames should be thought of as graphs. Following this we give the standard translation. This enables us to convert any basic modal formula into a first-order formula with one free variable. The standard translation is a bridge between the modal and classical worlds, a bridge that underlies much of the work of this chapter.

2.1 First steps in relational semantics

Syntax

Metalanguage

What follows we tacitly assume that $PROP$ is denumerably infinite, and we'll often work with signatures in which MOD contains only a single element. Given a signature, we define the *basic modal language* (over the signature) as follows:

$$\varphi ::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \mid \langle m \rangle \varphi \mid [m] \varphi.$$

That is, a basic modal formula is either a proposition symbol, a boolean constant, a boolean combination of basic modal formulas, or (most interesting of all) a formula prefixed by a diamond

A *model* (or *Kripke model*) $\mathfrak{M}$ for the basic modal language (over some fixed signature) is a triple $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Here W , the *domain*, is a non-empty set, whose elements we usually call *points*, but which, for reasons which will soon be clear, are sometimes called *states*, *times*, *situations*, *worlds* and other things besides. Each R^m in a model is a binary relation on W , and V is a function (the valuation) that assigns to each proposition symbol p in PROP a subset $V(p)$ of W ; think of $V(p)$ as the set of points in $\mathfrak{M}$ where p is true. The first two components $(W, \{R^m\}_{m \in \text{MOD}})$ of $\mathfrak{M}$ are called the *frame* underlying the model. If there is only one relation in the model, we typically write (W, R) for its frame, and (W, R, V) for the model itself. We encourage the reader to think of Kripke models as graphs (or to be slightly more precise, *directed graphs*, that is, graphs whose points are linked by directed arrows) and will shortly give some examples which show why this is helpful.

Suppose w is a point in a model $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Then we inductively define the notion of a formula φ being *satisfied* (or *true*) in $\mathfrak{M}$ at point w as follows (we omit some of the clauses for the booleans):

$\mathfrak{M}, w \models p$	iff	$w \in V(p)$,
$\mathfrak{M}, w \models \top$		always,
$\mathfrak{M}, w \models \perp$		never,
$\mathfrak{M}, w \models \neg \varphi$	iff	not $\mathfrak{M}, w \models \varphi$ (notation: $\mathfrak{M}, w \not\models \varphi$),
$\mathfrak{M}, w \models \varphi \wedge \psi$	iff	$\mathfrak{M}, w \models \varphi$ and $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \varphi \rightarrow \psi$	iff	$\mathfrak{M}, w \not\models \varphi$ or $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \langle m \rangle \varphi$	iff	for some $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$,
$\mathfrak{M}, w \models [m] \varphi$	iff	for all $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$.

A *model* (or *Kripke model*) $\mathfrak{M}$ for the basic modal language (over some fixed signature) is a triple $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Here W , the *domain*, is a non-empty set, whose elements we usually call *points*, but which, for reasons which will soon be clear, are sometimes called *states*, *times*

and V
 $V(p)$
 $(W, \{$
 in the

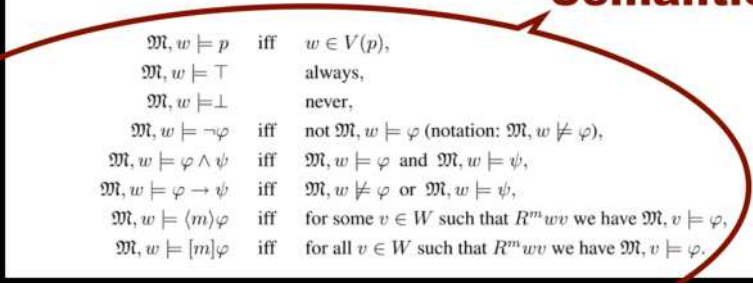
Metalanguage

in a model is a binary relation on W ,
 position symbol p in PROP a subset
 p is true. The first two components
 : model. If there is only one relation
 (W, R, V) for the model itself. We

encourage the reader to think of Kripke models as graphs (or to be slightly more precise, *directed graphs*, that is, graphs whose points are linked by directed arrows) and will shortly give some examples which show why this is helpful.

Suppose w is a point in a model $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Then we inductively define the notion of a formula φ being *satisfied* (or *true*) in $\mathfrak{M}$ at point w as follows (we omit some of the clauses for the booleans):

Semantics



$\mathfrak{M}, w \models p$	iff	$w \in V(p)$,
$\mathfrak{M}, w \models \top$		always,
$\mathfrak{M}, w \models \perp$		never,
$\mathfrak{M}, w \models \neg \varphi$	iff	not $\mathfrak{M}, w \models \varphi$ (notation: $\mathfrak{M}, w \not\models \varphi$),
$\mathfrak{M}, w \models \varphi \wedge \psi$	iff	$\mathfrak{M}, w \models \varphi$ and $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \varphi \rightarrow \psi$	iff	$\mathfrak{M}, w \not\models \varphi$ or $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \langle m \rangle \varphi$	iff	for some $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$,
$\mathfrak{M}, w \models [m] \varphi$	iff	for all $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$.

Kripke Style Semantics

$M, g, s \models P$ if and only if $s \in g(P)$

$M, g, s \models \neg \varphi$ if and only if $M, g, s \not\models \varphi$

$M, g, s \models \varphi \vee \psi$ if and only if $M, g, s \models \varphi$ or $M, g, s \models \psi$

$M, g, s \models \Box \varphi$ if and only if for all t with sRt we have $M, g, t \models \varphi$

Kripke Style Semantics

$M, g, s \models P$ if and only if $s \in g(P)$

$M, g, s \models \neg \varphi$ if and only if $M, g, s \not\models \varphi$

$M, g, s \models \varphi \vee \psi$ if and only if $M, g, s \models \varphi$ or $M, g, s \models \psi$

$M, g, s \models \Box \varphi$ if and only if for all t with sRt we have $M, g, t \models \varphi$

Standard Translation for Propositional Fragment (encoded in HOL)

- ▶ "lifted" predicate $P_{i \rightarrow o}$
- ▶ $\neg_{(i \rightarrow o) \rightarrow (i \rightarrow o)} = \lambda \varphi_{i \rightarrow o} \lambda w_i \neg(\varphi w)$
- ▶ $\vee_{(i \rightarrow o) \rightarrow (i \rightarrow o) \rightarrow (i \rightarrow o)} = \lambda \varphi_{i \rightarrow o} \lambda \psi_{i \rightarrow o} \lambda w_i \varphi w \vee \psi w$
- ▶ $\Box_{(i \rightarrow o) \rightarrow (i \rightarrow o)} = \lambda \varphi_{i \rightarrow o} \lambda w_i \forall v_i R w v \rightarrow \varphi v$

Validity

- ▶ $[\varphi_{i \rightarrow o}] = \forall w_i \varphi w$ resp. $[\cdot] = \lambda \varphi_{i \rightarrow o} \forall w_i \varphi w$

Modal Logic (in fact, Hybrid Logic) as a Fragment of HOL

Kripke Style Semantics

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ in HOL $\forall x_\alpha \varphi x$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha \varphi x)$ —no binder needed!!!
- ▶ $\Pi_{(\alpha \rightarrow (i \rightarrow o)) \rightarrow (i \rightarrow o)} = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work)

$$\begin{aligned} [\Box \forall x Px] &\equiv [\Box \Pi(\lambda x Px)] \equiv [\Box \Pi(\lambda x \lambda w Pxw)] \\ &\equiv [\Box ((\lambda \Phi \lambda w \Pi(\lambda x \Phi xw))(\lambda x \lambda w Pxw))] \\ &\equiv [\Box (\lambda w \Pi(\lambda x (\lambda w Pxw)xw))] \\ &\equiv [\Box (\lambda w \Pi(\lambda x Pxw))] \\ &\equiv [(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Pxw))] \\ &\equiv [(\lambda w \Pi(\lambda v R w v \rightarrow (\lambda w \Pi(\lambda x Pxw))v))] \\ &\equiv [\lambda w \Pi(\lambda v R w v \rightarrow \Pi(\lambda x Pxv))] \\ &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Pxv)] \\ &\equiv \forall w \forall v (R w v \rightarrow \forall x Pxv) \end{aligned}$$

- ▶ above: possibilist quantification
- ▶ actualist quantification: $\Pi = \lambda \Phi \lambda w \Pi(\lambda x \text{ existsAt } x \ w \rightarrow \Phi x w)$
- ▶ also supported: local and global validity and consequence

Kripke Style Semantics

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ in HOL $\forall x_\alpha \varphi x$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \varphi x)$ —no binder needed!!!
- ▶ $\Pi_{(\alpha \rightarrow (i \rightarrow o)) \rightarrow (i \rightarrow o)} = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \Pi(\lambda x Px)] &\equiv [\Box \Pi(\lambda x \lambda w Pxw)] \\
 &\equiv [\Box((\lambda \Phi \lambda w \Pi(\lambda x \Phi xw))(\lambda x \lambda w Pxw))] \\
 &\equiv [\Box(\lambda w \Pi(\lambda x (\lambda w Pxw)xw))] \\
 &\equiv [\Box(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda w \Pi(\lambda v R w v \rightarrow (\lambda w \Pi(\lambda x Pxw))v))] \\
 &\equiv [\lambda w \Pi(\lambda v R w v \rightarrow \Pi(\lambda x Pxw))] \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Pxv)] \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Pxv)
 \end{aligned}$$

- ▶ above: possibilist quantification
- ▶ actualist quantification: $\Pi = \lambda \Phi \lambda w \Pi(\lambda x \text{ existsAt } x \ w \rightarrow \Phi x w)$
- ▶ also supported: local and global validity and consequence

Kripke Style Semantics

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ in HOL $\forall x_\alpha \varphi x$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \varphi x)$ —no binder needed!!!
- ▶ $\Pi_{(\alpha \rightarrow (i \rightarrow o)) \rightarrow (i \rightarrow o)} = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \Pi(\lambda x Px)] &\equiv [\Box \Pi(\lambda x \lambda w Pxw)] \\
 &\equiv [\Box ((\lambda \Phi \lambda w \Pi(\lambda x \Phi xw))(\lambda x \lambda w Pxw))] \\
 &\equiv [\Box (\lambda w \Pi(\lambda x (\lambda w Pxw)xw))] \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda w \Pi(\lambda v R w v \rightarrow (\lambda w \Pi(\lambda x Pxw))v))] \\
 &\equiv [\lambda w \Pi(\lambda v R w v \rightarrow \Pi(\lambda x Pxv))] \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Pxv)] \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Pxv)
 \end{aligned}$$

- ▶ above: possibilist quantification
- ▶ actualist quantification: $\Pi = \lambda \Phi \lambda w \Pi(\lambda x \text{ existsAt } x \ w \rightarrow \Phi x w)$
- ▶ also supported: local and global validity and consequence

Kripke Style Semantics

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ in HOL $\forall x_\alpha \varphi x$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \varphi x)$ —no binder needed!!!
- ▶ $\Pi_{(\alpha \rightarrow (i \rightarrow o)) \rightarrow (i \rightarrow o)} = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \Pi(\lambda x Px)] &\equiv [\Box \Pi(\lambda x \lambda w Pxw)] \\
 &\equiv [\Box((\lambda \Phi \lambda w \Pi(\lambda x \Phi xw))(\lambda x \lambda w Pxw))] \\
 &\equiv [\Box(\lambda w \Pi(\lambda x (\lambda w Pxw)xw))] \\
 &\equiv [\Box(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda w \Pi(\lambda v R w v \rightarrow (\lambda w \Pi(\lambda x Pxw))v))] \\
 &\equiv [\lambda w \Pi(\lambda v R w v \rightarrow \Pi(\lambda x Pxw))] \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Pxv)] \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Pxv)
 \end{aligned}$$

- ▶ above: possibilist quantification
- ▶ actualist quantification: $\Pi = \lambda \Phi \lambda w \Pi(\lambda x \text{ existsAt } x \ w \rightarrow \Phi x w)$
- ▶ also supported: local and global validity and consequence

Kripke Style Semantics

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

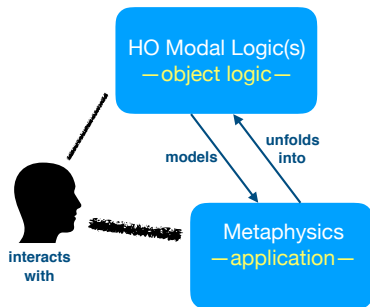
- ▶ in HOL $\forall x_\alpha \varphi x$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \varphi x)$ —no binder needed!!!
- ▶ $\Pi_{(\alpha \rightarrow (i \rightarrow o)) \rightarrow (i \rightarrow o)} = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

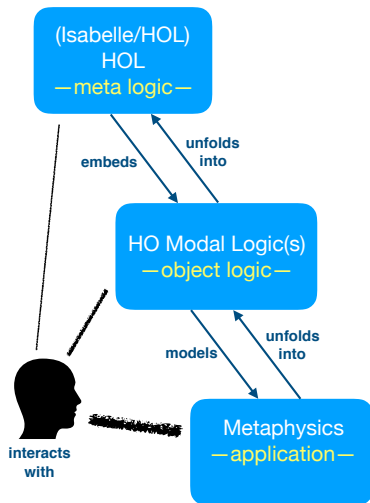
Example (compositionality and λ -conversion at work)

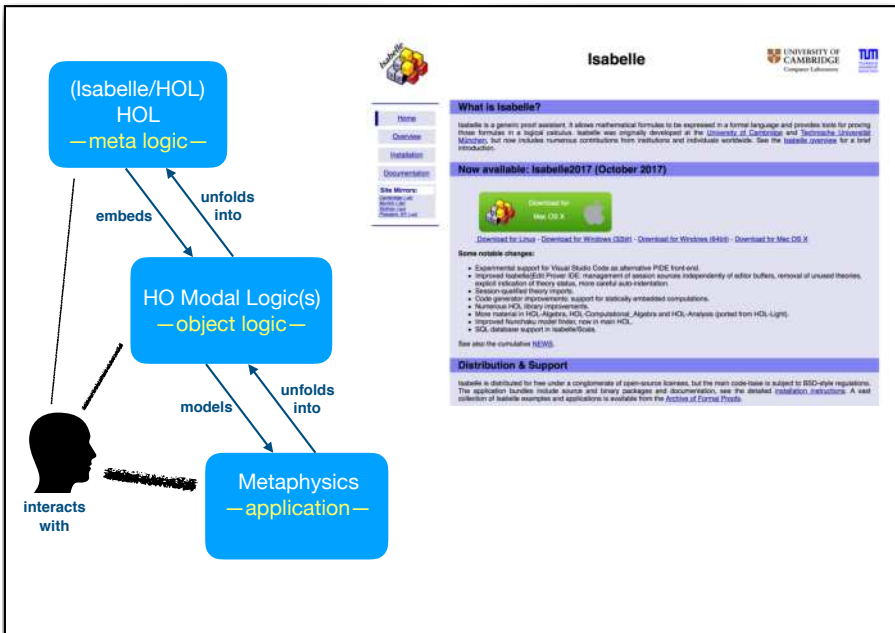
$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \Pi(\lambda x Px)] &\equiv [\Box \Pi(\lambda x \lambda w Pxw)] \\
 &\equiv [\Box((\lambda \Phi \lambda w \Pi(\lambda x \Phi xw))(\lambda x \lambda w Pxw))] \\
 &\equiv [\Box(\lambda w \Pi(\lambda x (\lambda w Pxw)xw))] \\
 &\equiv [\Box(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Pxw))] \\
 &\equiv [(\lambda w \Pi(\lambda v R w v \rightarrow (\lambda w \Pi(\lambda x Pxw))v))] \\
 &\equiv [\lambda w \Pi(\lambda v R w v \rightarrow \Pi(\lambda x Pxw))] \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Pxv)] \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Pxv)
 \end{aligned}$$

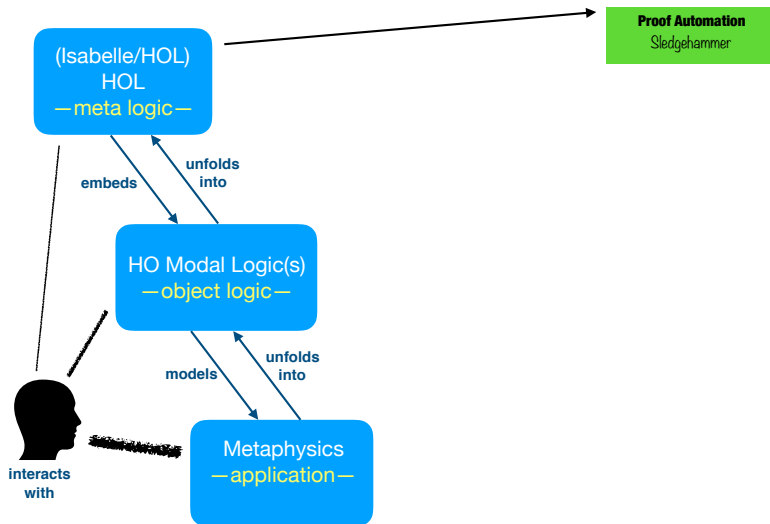
- ▶ above: possibilist quantification
- ▶ actualist quantification: $\Pi = \lambda \Phi \lambda w \Pi(\lambda x \text{ existsAt } x \ w \rightarrow \Phi x w)$
- ▶ also supported: local and global validity and consequence

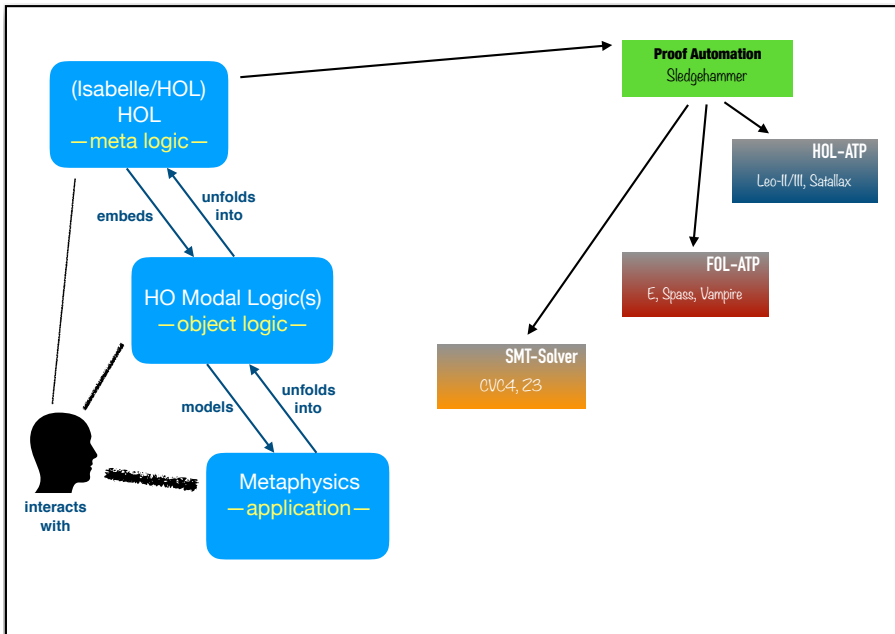


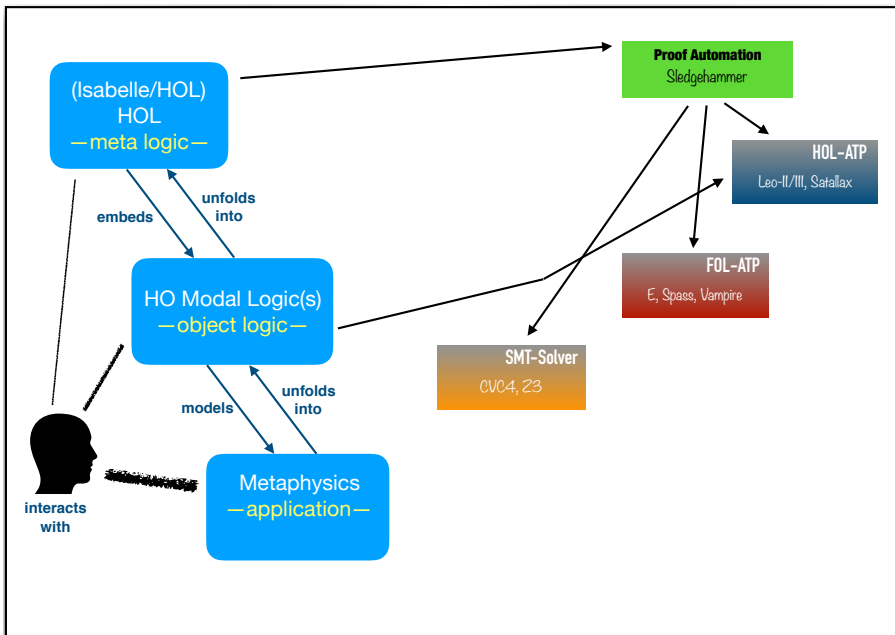


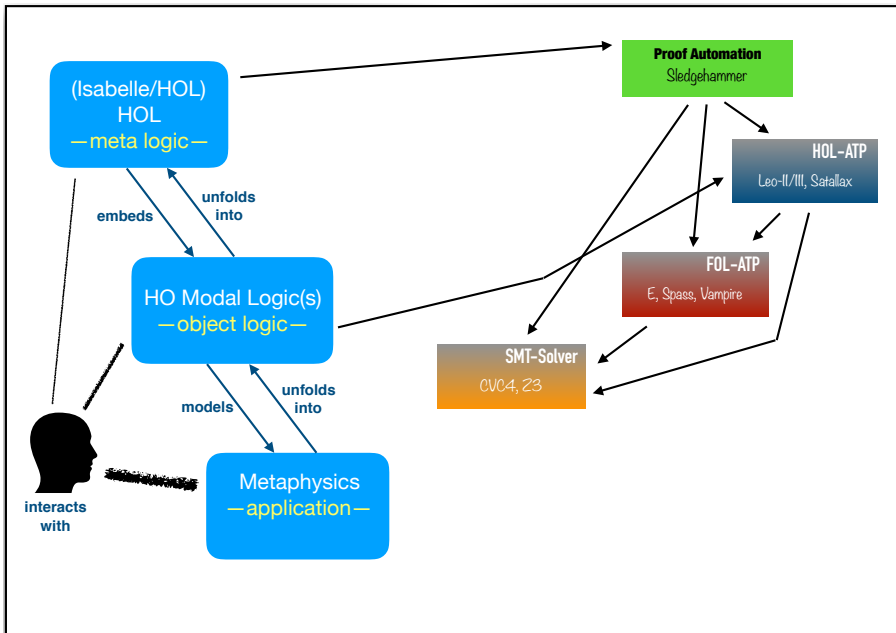


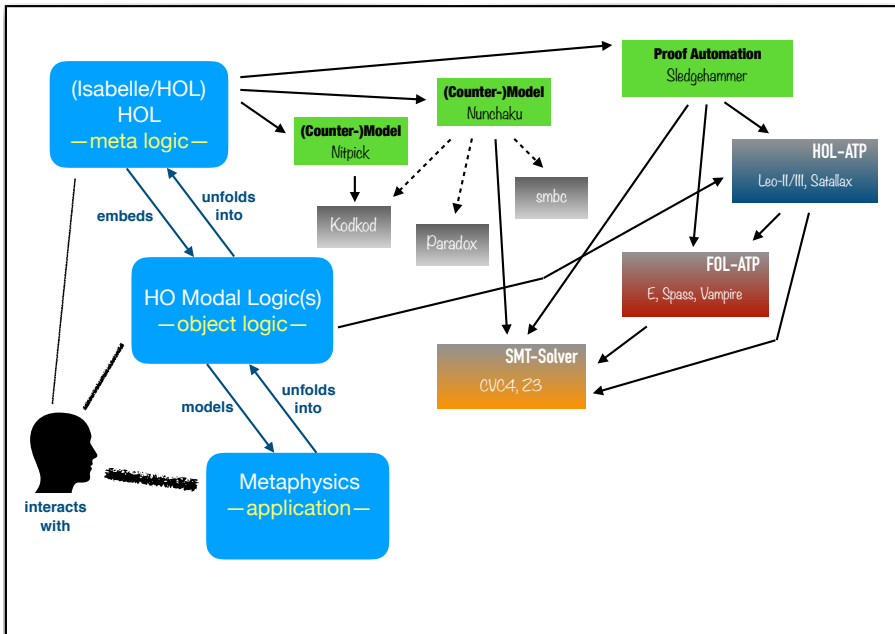


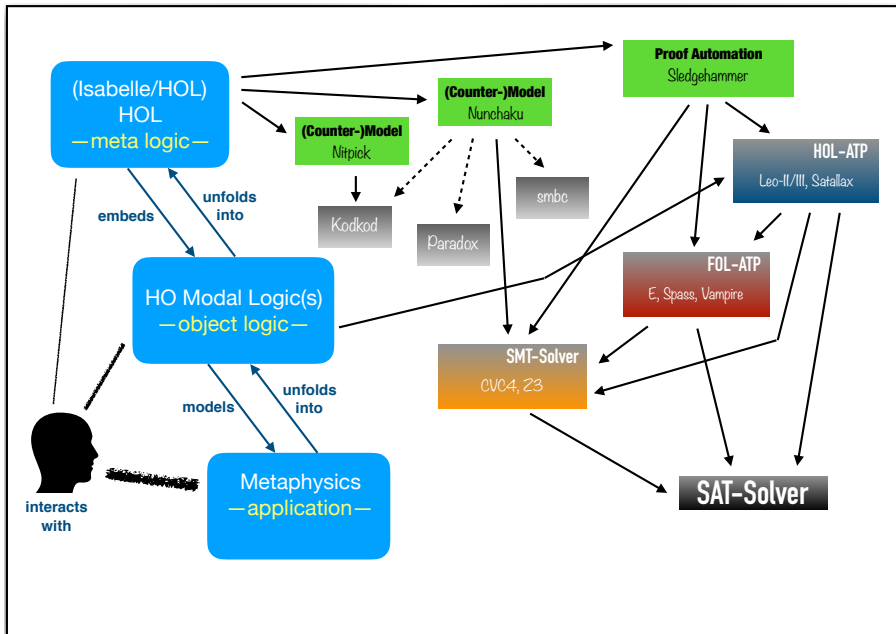


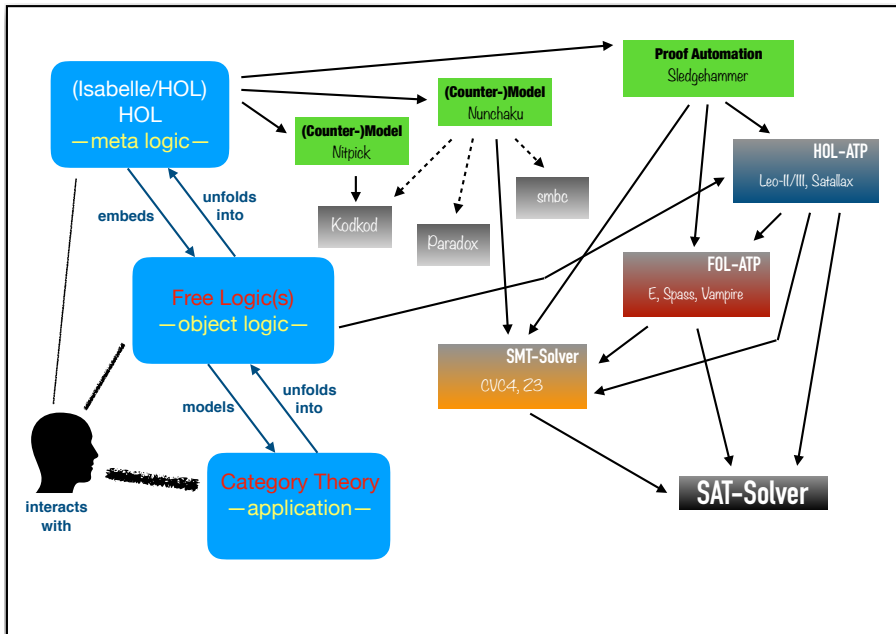


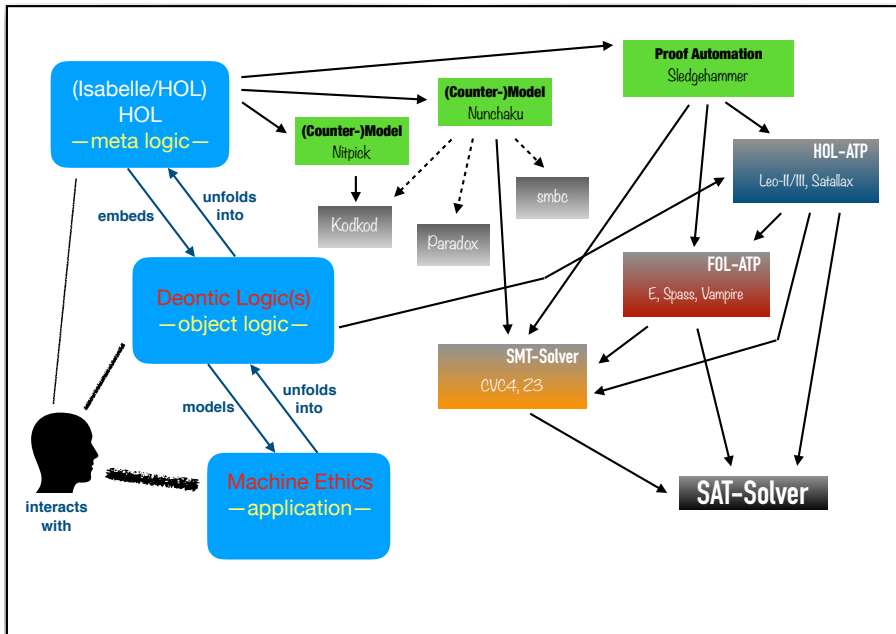


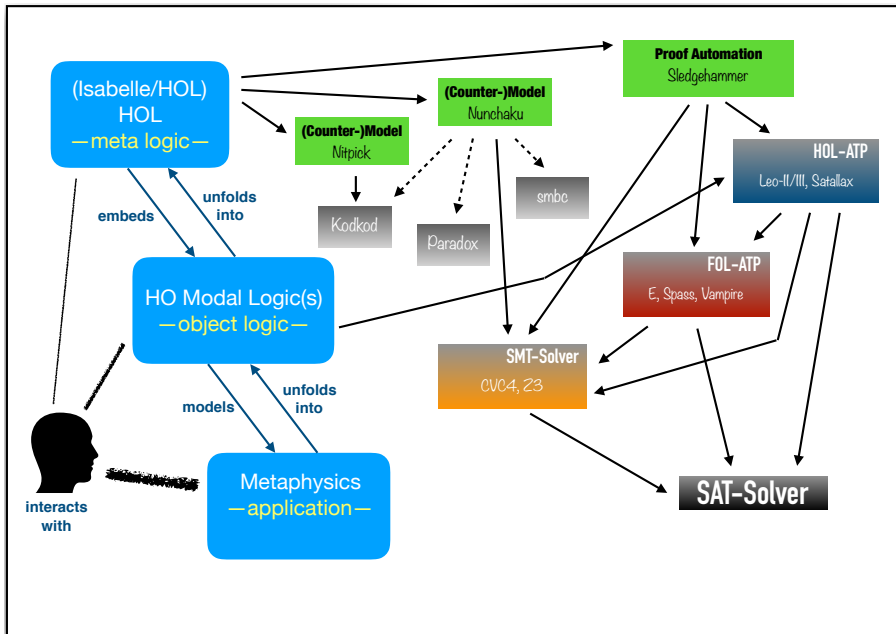












- ▶ **Philosophy:** Ontological Argument & Principia Metaphysica
[Open Philosophy, Review of Symbolic Logic, Savijnanam, Logica Universalis, IJCAI, ECAI]
- ▶ **Mathematics:** Category Theory and Free Logic
[Journal of Automated Reasoning, AITP, ICMS]
- ▶ **Artificial Intelligence:** Challenges in KR&R (e.g., Wise Men Puzzle)
[Science of Computer Programming, Data in Brief, Journal of Philosophical Logic, Annals of Mathematics and Artificial Intelligence, IJCAI]
- ▶ **Natural Language:** Computational Hermeneutics, Argumentation
[Journal of Applied Logic, Sapere, CLAR, MBR]
- ▶ **Machine Ethics:** Normative Reasoning, Deontic Logics, Ethical Theories
[Journal of Applied Logics, DEON, CiE, FroCoS, PRICAI]

... more to come ...

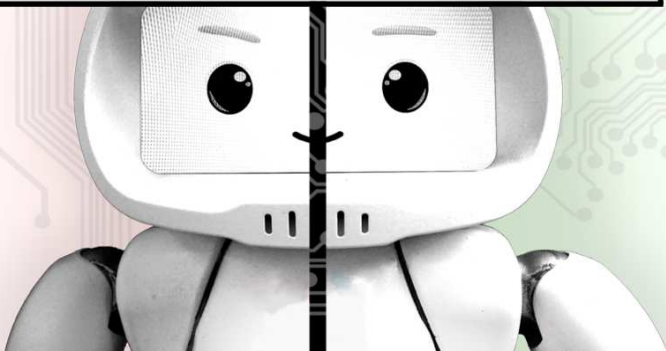
— Part III —

Thrustworthy & Explainable AI, Machine Ethics

Ethics

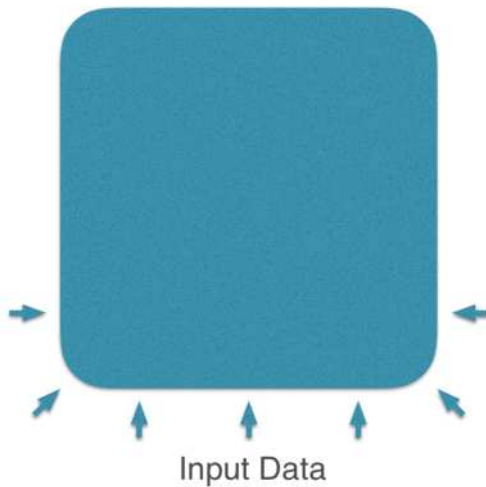
“If we had it [a *characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”

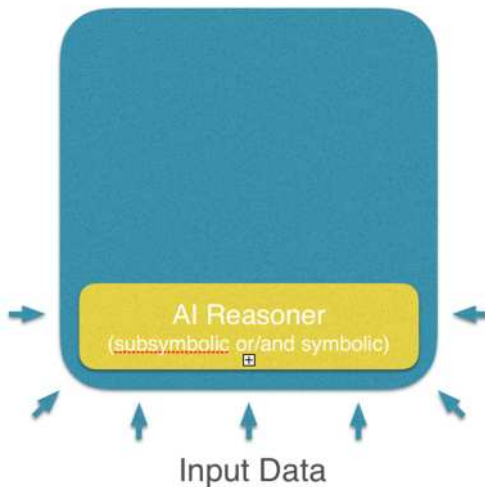
(Leibniz, 1677)

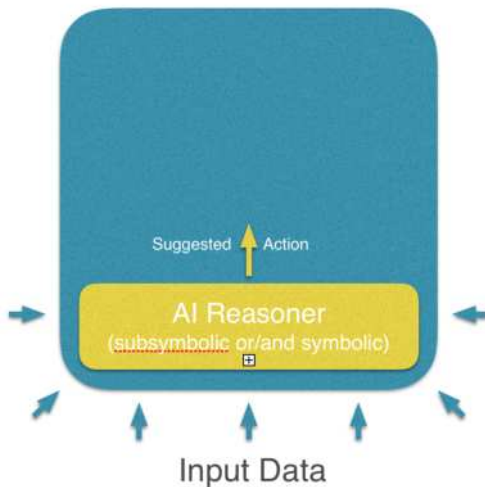


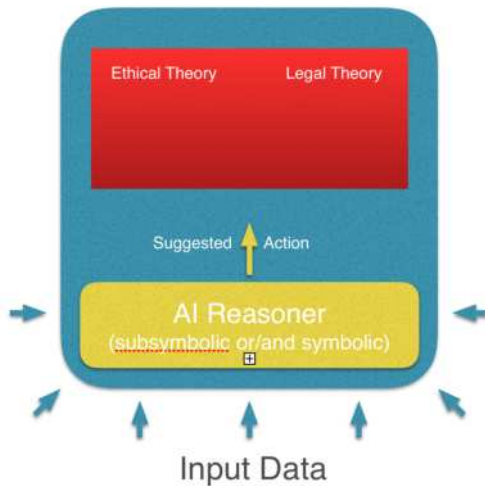
A large teal rounded square with the letters "IAS" in white, centered within it.

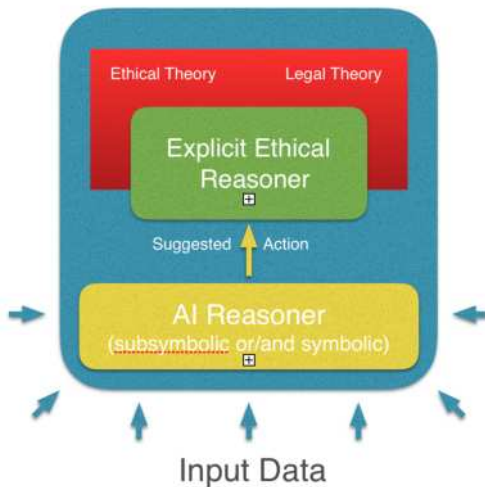
IAS

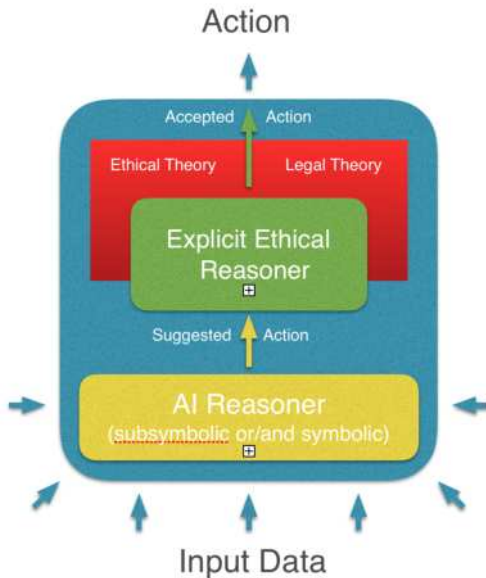


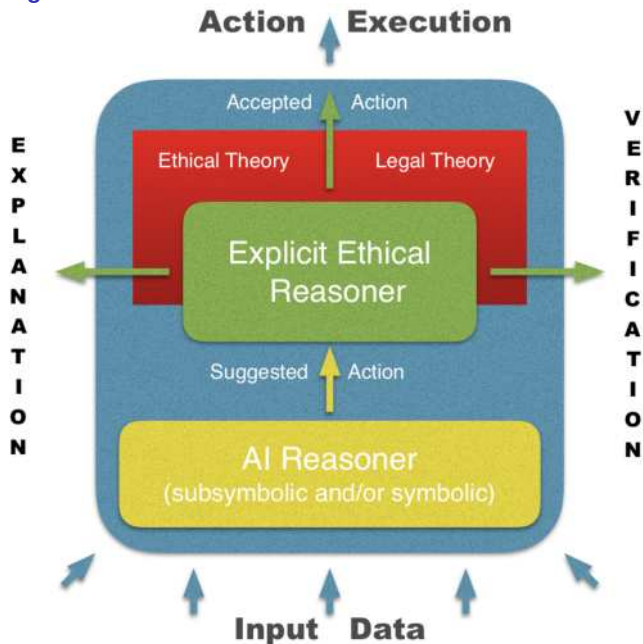


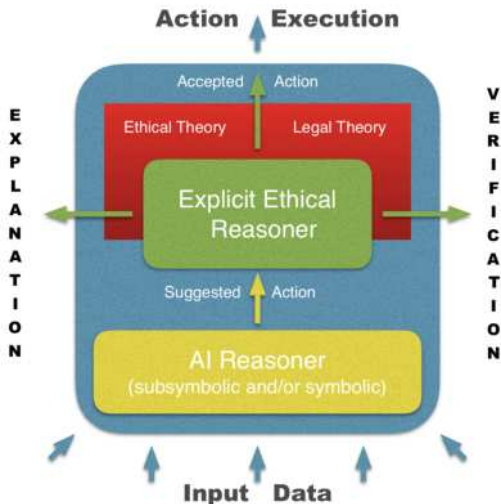






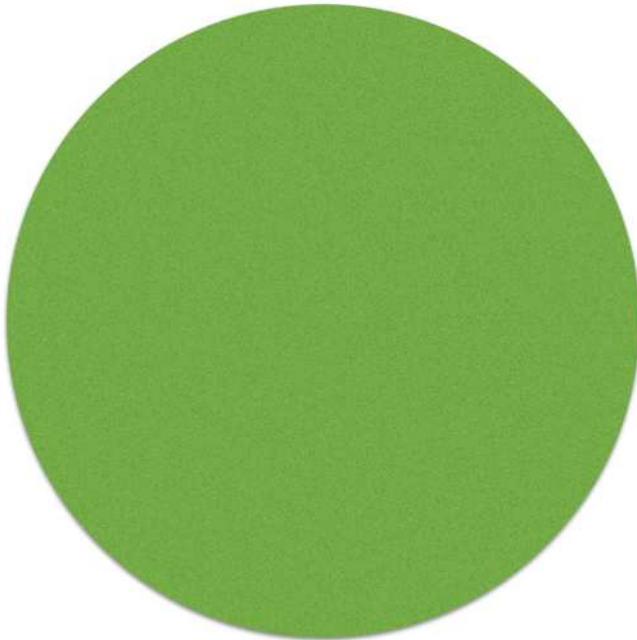


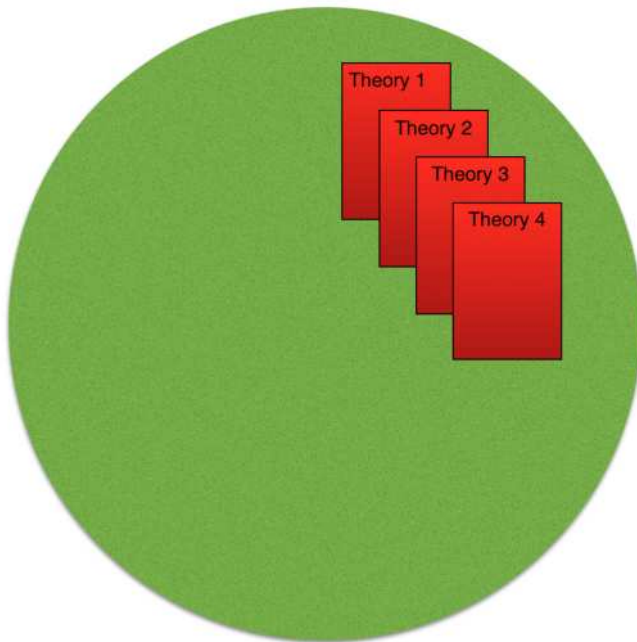


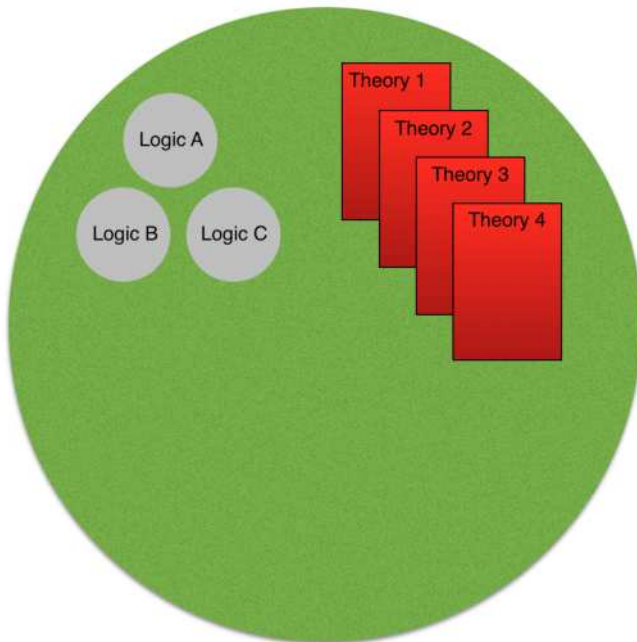


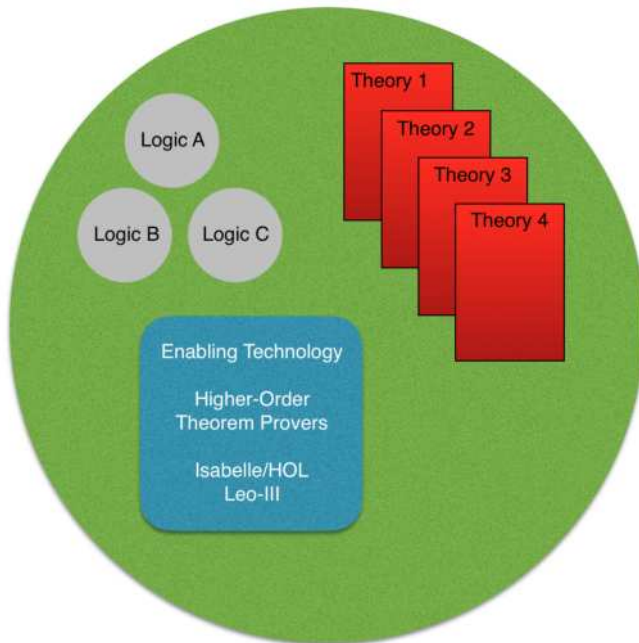
Related Work

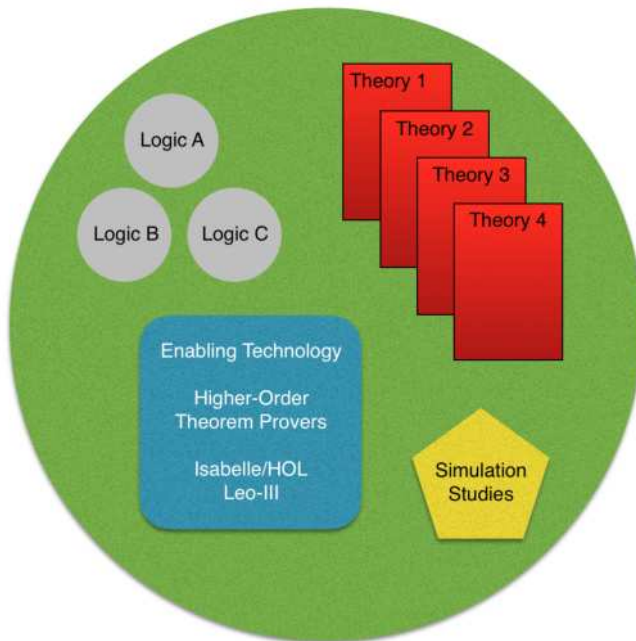
- ▶ Toward Ethical Robots
 - ▶ [ArkoudasEtAl., 2005]
- ▶ Artificial Moral Agents
 - ▶ [Wallach&Allen, 2008]
- ▶ Ethical Governors
 - ▶ [ArkinEtAl., 2009, 2012]
 - ▶ [Dennis&Fisher, 2017]
- ▶ Ethical Deliberation in ART
 - ▶ [Dignum, 2017]
- ▶ Programming Machine Ethics
 - ▶ [Pereira&Saptawijaya, 2016]
- ▶ ...



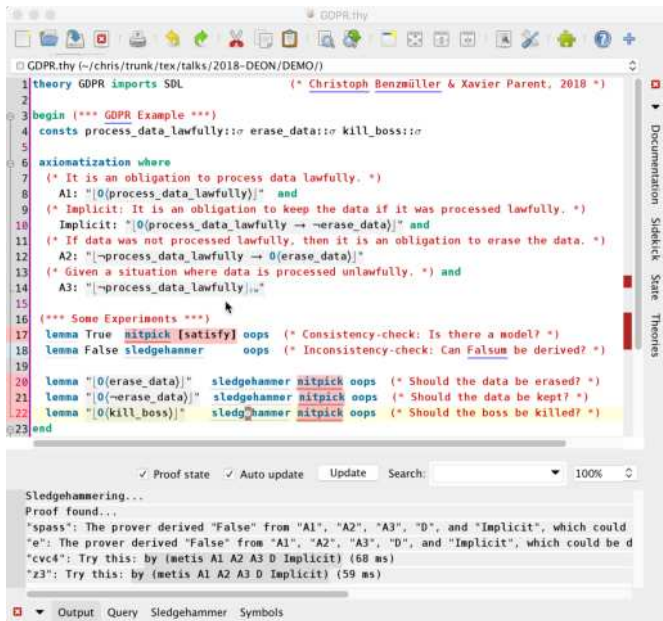








Flexible normative reasoning and deontic logics — DEMO in Isabelle/HOL



```
1 theory GDPR imports SDL (* Christoph Benzmüller & Xavier Parent, 2018 *)
2
3 begin (** GDPR Example **)
4 consts process_data_lawfully:: $\sigma$  erase_data:: $\sigma$  kill_boss:: $\sigma$ 
5
6 axiomatization where
7 (* It is an obligation to process data lawfully. *)
8 A1: "[0(process_data_lawfully)]" and
9 (* Implicit: It is an obligation to keep the data if it was processed lawfully. *)
10 Implicit: "[0(process_data_lawfully  $\rightarrow$   $\neg$ erase_data)]" and
11 (* If data was not processed lawfully, then it is an obligation to erase the data. *)
12 A2: "[ $\neg$ process_data_lawfully  $\rightarrow$  0(erase_data)]"
13 (* Given a situation where data is processed unlawfully. *) and
14 A3: "[ $\neg$ process_data_lawfully].."
15
16 (** Some Experiments **)
17 lemma True nitpick [satisfy] oops (* Consistency-check: Is there a model? *)
18 lemma False sledgehammer oops (* Inconsistency-check: Can Falsum be derived? *)
19
20 lemma "[0(erase_data)]" sledgehammer nitpick oops (* Should the data be erased? *)
21 lemma "[0( $\neg$ erase_data)]" sledgehammer nitpick oops (* Should the data be kept? *)
22 lemma "[0(kill_boss)]" sledgehammer nitpick oops (* Should the boss be killed? *)
23 end
```

✓ Proof state ✓ Auto update Update Search: 100%

Sledgehammering...
Proof found...
"spass": The prover derived "False" from "A1", "A2", "A3", "D", and "Implicit", which could
"e": The prover derived "False" from "A1", "A2", "A3", "D", and "Implicit", which could be d
"cvc4": Try this: by (metis A1 A2 A3 D Implicit) (68 ms)
"z3": Try this: by (metis A1 A2 A3 D Implicit) (59 ms)

Output Query Sledgehammer Symbols

Flexible normative reasoning and deontic logics — DEMO in Isabelle/HOL

GDPR.thy (~chris/trunk/tex/talks/2018-DEON/DEMO/)

```
1 theory GDPR imports SDL (* Christoph Benzmüller & Xavier Parent, 2018 *)
2
3 begin (** GDPR Example **)
4 consts process_data_lawfully::o erase_data::o kill_boss::o
5
6 axiomatization where
7 (* It is an obligation to process data lawfully. *)
8 A1: "[O(process_data_lawfully)]" and
9 (* Implicit: It is an obligation to keep the data if it was processed lawfully. *)
10 Implicit: "[O(process_data_lawfully → ¬erase_data)]" and
11 (* If data was not processed lawfully, then it is an obligation to erase the data. *)
12 A2: "[¬process_data_lawfully → O(erase_data)]"
13 (* Given a situation where data is processed unlawfully. *) and
14 A3: "[¬process_data_lawfully]..".
15
16 (*
17
18
19
20
21 lemma "[O(¬erase_data)]" sledgehammer nitpick oops (* Should the data be kept? *)
22 lemma "[O(kill_boss)]" sledgehammer nitpick oops (* Should the boss be killed? *)
23 end
```

**Danger Zone:
Paradoxes and Inconsistencies!**

✓ Proof state ✓ Auto update Update Search: 100%

Sledgehammering...
Proof found...
"spass": The prover derived "False" from "A1", "A2", "A3", "D", and "Implicit", which could
"e": The prover derived "False" from "A1", "A2", "A3", "D", and "Implicit", which could be d
"cvc4": Try this: by (metis A1 A2 A3 D Implicit) (68 ms)
"z3": Try this: by (metis A1 A2 A3 D Implicit) (59 ms)

Output Query Sledgehammer Symbols

“Act in accord with the generic rights of your recipients as well as of yourself. I shall call this the **Principle of Generic Consistency (PGC)**, since it combines the formal consideration of consistency with the material consideration of rights to the generic features or goods of action.”
(Alan Gewirth, **Reason and Morality**, 1978)

▶ **Gewirth's PGC:**

- ▶ stirred controversy in moral philosophy
- ▶ discussed as means to bound the impact of artificial general intelligence (AGI)

▶ **Idea (in a nutshell):**

- ▶ devise a safety mechanism of a mathematical (deductive) nature
- ▶ to ensure that an AGI respects human's freedom and well-being
- ▶ based on assumption that agents (AGI&Humans) reason rationally

▶ **References**

- ▶ A. Gewirth. Reason and morality. UCP, 1978.
- ▶ D. Beyleveld. The dialectical necessity of morality: An analysis and defense of Alan Gewirth's argument to the principle of generic consistency. UCP, 1991.
- ▶ A. Kornai. Bounding the impact of AGI. J. Experimental & Theoretical AI, 2014.

Mechanization of the PGC presented at FroCoS & PRICAI this year.

Conclusion

From
 traditional reasoning about (imperative) programs
via
 reasoning about agents & smart contracts
to
 reasoning about ethico-legal constraints for intelligent agents

All within a single approach:

Universal meta-logical reasoning in HOL

Very useful for scientific exploration, rapid-prototyping and teaching!

“If we had it [a *characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”

(Leibniz, 1677)

Universal Meta-Logical Reasoning in HOL

- ▶ Foundations & Limits
- ▶ Automation & Interaction
- ▶ Applications: CS, AI, Maths, Philosophy, NL

Vision and Plans: More Universal Meta-Logical Reasoning

- ▶ **CS:** automated program reasoning
- ▶ **AI:** normative reasoning and explicit ethical governors, KR&R
- ▶ **Maths:** foundations, category theory, type theory, set theory
- ▶ **Philosophy:** metaphysics, rational argumentation, ontologies
- ▶ **NL:** deep semantical analysis

Numerous recent keynotes in CS, AI, Maths and Philosophy



Vision and Plans: Foster International Collaborations

Universal Logical Reasoning and Computational Metaphysics:

• Edward Zalta, Stanford U, US • Walter Carnielli, U Campinas, BR • Bruno Woltzenlogel-Paleo, AU

Normative Reasoning and Ethical Intelligent Systems:

• Leon van der Torre, Xavier Parent, Martin Theobald, U Luxembourg, LU • Toby Walsh, UNSW Sydney, AU • Beishui Liao, Zhejiang U, CH

History and AI: • Jessica Gienow-Hecht, JFK-Institut, FU Berlin, DE

Automated and Interactive Theorem Proving:

• Geoff Sutcliffe, U Miami, US • Jasmin Blanchette, Vrije U Amsterdam, NL • Stephan Schulz, DHBW Stuttgart, DE • Larry Paulson, U Cambridge, GB • Jens Otten, U Oslo, NO • Chad Brown, Czech TU in Prague, CZ • Frank Pfenning, CMU, US

Robotics and Machine Learning: • Raul Rojas, Daniel Göhrling, FU Berlin

AI Ontologies: • Adam Pease, InfoSys, US

Education Tools: • Carsten Ullrich, DFKI Berlin, DE

Foundations of Maths: • Dana Scott, U Berkeley, CA, US

Contact point in the CLAIRE (<https://claire-ai.org>) network

Vision and Plans: Foster Collaborations within TU Vienna

Connections to TU Vienna include:

- ▶ 2018: **Hanna Lachnitt**, Helmut-Veith-Award for her BSc thesis
- ▶ 2017: **Martin Riener**, Doctorate, CS, Applications of HO Cut-Elimination, committee member, supervisor A. Leitsch
- ▶ 2017: **Hans-Jörg Schurr**, Masters, CS, Preprocessing in Higher-order Reasoning – Learning from QBF Solving, co-supervisor with A. Leitsch

Vision and Plans: Interdisciplinary Research&Education – Example

Who is Driving the Future? Autonomous Vehicles & AI in the Frame of History (AIH)

Freie Universität Berlin and Carleton University, Ottawa

Duration: 3 years
Budget: 1.499.000 €

Jessica Gienow-Hecht
Freie Universität Berlin
John F. Kennedy Institute
Department of History
Lansstraße 7-9
Raum 216
14195 Berlin, DE

Christoph Benz Müller
Freie Universität Berlin
Dahlem Center for Machine Learning and Robotics
Animalsee 7
Raum 115
14195 Berlin, DE

Andrew Johnston
Carleton University
Department of History
414 Paterson Hall
1125 Colonel By Drive
Ottawa, ON, K1S 5B6, CA

Daniel Göhring
Freie Universität Berlin
Dahlem Center for Machine Learning and Robotics
Animalsee 7
Room E29
14195 Berlin, DE

Jim Davies
Carleton University
Institute of Cognitive Science
2208 Dunton Tower
1125 Colonel By Drive
Ottawa, ON, K1S 5B6, CA

Vision and Plans: Interdisciplinary Research&Education – Example

Who is Driving the Future? Autonomous Vehicles & AI in the Frame of History (AIH)

Freie Universität Berlin and Carleton University, Ottawa

Duration: 3 years
Budget: 1.499.000 €

Jessica Gienow-Hecht
Freie Universität Berlin
John F. Kennedy Institute
Department of History
Lansstraße 7-9
Raum 216
14195 Berlin, DE

Christoph Benz Müller
Freie Universität Berlin
Dahlem Center for
Mammalogy 7
Raum 115
14195 Berlin, DE

Andrew Johnston
Carleton University
Department of History
414 Paterson Hall
1125 Colonel By Drive
Ottawa, ON, K1S 5B6

Daniel Göhring
Freie Universität Berlin
Dahlem Center for
Mammalogy 7
Room E29
14195 Berlin, DE

Jim Davies
Carleton University
Institute of Cognitive
2208 Dunton Tower
1125 Colonel By Drive
Ottawa, ON, K1S 5B6

How can scientists, policymakers, and manufacturers draw on historical precedents of disruptive technological transformations to address the socio-cultural and ethical challenges of current AI technology such as autonomous vehicles? The project will examine this question in relation to one particular case study, the driverless car, which will potentially have the greatest impact on work, leisure, and education, and thus offers a cross-section of the larger social and technical issues tied up with AI today. Testing Z. B. Simon's theory of unprecedented historical change, the project will relate contemporary AV research with historical data regarding how previous regulatory regimes responded to the challenge of automobility. A knowledge exchange lab will integrate research around four key concerns informing AI today: (1) human-computer-interaction, (2) increased automation, (3) control and regulation, (4) ethics and privacy. The project will generate: a set of historically-informed AI data and analyses that will enable governments, manufacturers, and consumers to make informed decisions about AV adoption; implementation recommendations for national and supranational decision makers; high-caliber peer-reviewed papers; a volume planned for Berghahn Books (Oxford/New York) that challenges Simon's theory; an online portal, including a digitally interactive world map surveying planned, current, and historical key moments in AI technological development.

Vision and Plans: Foster interdisciplinary profile of students

Alexander Steen (now PostDoc at U Luxembourg)

- ▶ B.A. in CS and in Maths, MSc in CS,
- ▶ Most active student (tutor & committees & boards) at FU Berlin
- ▶ PhD in CS (summa cum laude, world champion with Leo-III)
- ▶ GI-Junior-Fellowship in 2018, nominated for further awards

David Fuenmayor

- ▶ B.A. in Mechatronics (Bogota and Karlsruhe), B.A. Philosophy/Anthropology
- ▶ Works in cloud computing: CGI Group (senior consultant), SAP Research
- ▶ PhD student in Computer Science/Philosophy (Topic: Computational Hermeneutics and Rational Argumentation)
- ▶ Publications: ≥ 12 articles and conference papers in 2,5 years
- ▶ currently visiting Walter Carnielli (UNICAMP, BR)

Daniel Kirchner

- ▶ B.A. and MSc in Mathematics (FU Berlin)
- ▶ Works for Ethereum (Smart Contracts)
- ▶ PhD student in Mathematics/CS/Philosophy (Topic: Formalization of Principia Metaphysica)
- ▶ Publications: Review of Symbolic Logic, Open Philosophy

Vision and Plans: Interdisciplinary Courses – Example

Freie Universität BerlinQ ...

campus.leben

MENÜ☰

Startseite > campus.leben > LERNEN & LEHREN > 2016 > Forschen lernen im digitalen Zeitalter

Forschen lernen im digitalen Zeitalter

Projekt von Informatiker Christoph Benz Müller mit dem Lehrpreis der Freien Universität Berlin ausgezeichnet

02.05.2016

Die Entscheidung sei in diesem Jahr besonders schwergefallen, sagte Klaus Hoffmann-Holland, Vizepräsident für Forschung und Lehre der Freien Universität Berlin und Mitglied der Jury für den Lehrpreis, anlässlich der Preisverleihung. Deshalb wurde in diesem Jahr nicht nur ein herausragendes Lehrprojekt ausgezeichnet, es wurden auch zwei weitere gewürdigt. Wie groß die Vielfalt der Angebote zur forschungsorientierten Lehre an der Freien Universität ist, zeigte eine Podiumsdiskussion im Anschluss an die Preisverleihung.



Ausgezeichnet mit dem Lehrpreis: Informatiker Christoph Benz Müller.
Bildquelle: Manuel Krane

Vision and Plans: Systems & Textbooks

System development HOL-ATP(ongoing)

- ▶ Leo family of higher-order ATPs
- ▶ ATPs for quantified non-classical logics
- ▶ Infrastructure for ATP in these logics

Infrastructure for ATP in Quantified Non-Classical Logics

- ▶ planned with Geoff Sutcliffe

Textbooks (requested, in preparation)

- ▶ Computational Metaphysics
Variants of the ontological argument on the computer
Universal meta-logical reasoning
Intended audience: Interested researchers from all disciplines, wider public
- ▶ Expressive Classical and Non-Classical Logics and their Automation
Expressive classical and non-classical logics
Proof automation
Universal meta-logical reasoning
Intended audience: Undergraduate students

This creates excellent opportunities for student projects!

Vision and Plans: Teaching & Research

- ▶ **Expressive Logics**
 - ▶ syntax, semantics, abstract consistency, proof theory
 - ▶ program: logic and computation, software engineering and internet computing
- ▶ **Automated Deduction (focus on expressive logics)**
 - ▶ from SAT/SMT via FOL to HOL (& more)
 - ▶ cooperation with existing course on Automated Deduction
 - ▶ program: logic and computation, software engineering and internet computing
- ▶ **Automated Program Reasoning (maybe focus on contract languages)**
 - ▶ remember my teaching concepts
 - ▶ program: logic and computation, software engineering and internet computing
- ▶ **Artificial Intelligence (focus on trustworthy AI & machine ethics)**
 - ▶ KR&R, foundations, explainable & trustworthy AI, machine ethics, normative reasoning, multi-agent systems, ontologies
 - ▶ program: logic and computation, software engineering and internet computing
- ▶ **Computational Metaphysics** (awarded lecture course)
- ▶ **Proof Assistant Systems and ITP**
- ▶ **Lambda Calculus**
- ▶ **Functional Programming**
- ▶ **Type Theory**

Discussion & Questions

Which Reasoning Formalisms?

“If we had it [a *characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”

(Leibniz, 1677)

Challenges for Explicit Ethical Reasoning Engines: Which Logic(s)?

- ▶ Dilemmas, conflicting theories, etc.
- ▶ Appropriate modeling-of/reasoning-with notion of **obligation**
 - ▶ Contrary-to-duty (CTD) scenarios

Which Reasoning Formalisms?

“If we had it [a *characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”

(Leibniz, 1677)

Challenges for Explicit Ethical Reasoning Engines: Which Logic(s)?

- ▶ Dilemmas, conflicting theories, etc.
- ▶ Appropriate modeling-of/reasoning-with notion of **obligation**
 - ▶ Contrary-to-duty (CTD) scenarios

Standard CTD structure (Chisholm)

1. obligatory 'a'
 2. obligatory 'if *a* then not *b*'
 3. if 'not *a*' then obligatory '*b*'
 4. 'not *a*'
- (in a given situation)

Danger: Paradox/inconsistency — ex falso quodlibet!

Which Reasoning Formalisms?

“If we had it [a *characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”

(Leibniz, 1677)

Challenges for Explicit Ethical Reasoning Engines: Which Logic(s)?

- ▶ Dilemmas, conflicting theories, etc.
- ▶ Appropriate modeling-of/reasoning-with notion of **obligation**
 - ▶ Contrary-to-duty (CTD) scenarios

CTD example (X. Parent): **EU General Data Protection Regulation (GDPR)**

1. Personal data shall be processed lawfully. (Art. 5)
E.g., the data subject must have given consent to the processing. (Art. 6/1.a)
2. **Implicit:** The data shall be kept, for the agreed purposes, if processed lawfully.
3. If personal data has been processed unlawfully, the controller has the obligation to erase the personal data in question without delay. (Art. 17.d, right to be forgotten)
4. **Given situation:** Some personal data has been processed unlawfully.

Danger: Paradox/inconsistency — ex falso quodlibet!

Which Reasoning Formalisms?

“If we had it [a *characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”

(Leibniz, 1677)

Challenges for Explicit Ethical Reasoning Engines: Which Logic(s)?

- ▶ Dilemmas, conflicting theories, etc.
- ▶ Appropriate modeling-of/reasoning-with notion of **obligation**
 - ▶ Contrary-to-duty (CTD) scenarios

Deontic Logic

- ▶ Reasoning about obligations and permissions
- ▶ Two groups of approaches:
 - Possible worlds
 - ▶ standard deontic logic CTD: **no**
 - ▶ dyadic deontic logic CTD: **yes**
 - Norm-based semantics
 - ▶ input/output logic CTD: **yes**



L. van der Torre



X. Parent



A. Farjami

Further interests and challenges

- ▶ Combination with other logics (other modalities)
- ▶ Propositional deontic logic(s) will hardly be sufficient in practice

Bundesregierung (Nov 2018): Strategie Künstliche Intelligenz

*“Ethische und rechtsstaatliche Anforderungen sollten als integraler Bestandteil — und damit Markenzeichen einer „AI made in Europe“ — im gesamten Prozess der Entwicklung und Anwendung von KI Beachtung finden. Dies umfasst die Forschung, Entwicklung und die Produktion von KI, aber auch den Einsatz, den Betrieb, die **Kontrolle und die Governance KI-basierter Anwendungen**. Entwicklung von Verfahren zur Kontrolle und Nachvollziehbarkeit algorithmischer Entscheidungen sollte alle Akteure, inkl. Industrie, einbeziehen.”*

https://www.bmbf.de/files/Nationale_KI-Strategie.pdf; page 40

Ben Goertzel (CEO SingularityNET; Nov 2018): “Toward Democratic, Lawful Citizenship for AIs, Robots, and Corporations”

*“Being an effective citizen of a nation operating under rule of law requires a form of general intelligence that combines formal linguistic and symbolic knowledge (the legal code) with the ability to abstract patterns from multimodal sensory data and informal linguistic data (corresponding to actual real-life situations to which the law needs to be applied). So an AI Citizenship Test needs to be a particular form of a General Intelligence Test. And it needs to be a test that stresses one of the most interesting issues at the core of modern AI R&D: the **fusion of symbolic and subsymbolic knowledge**.”*

<https://tinyurl.com/y8h94ouv>